

FPGA 与 VHDL 考试 变式题解析册

2026 年 6 月 2 日

目录

第一部分 计数器变式题解析 (20 题)	3
第一章 计数器变式题 (20 题完整解析)	4
1.1 题 1: 模 7 计数器	4
1.2 题 2: 模 5 计数器	6
1.3 题 3: 模 12 计数器	7
1.4 题 4: 模 13 计数器	8
1.5 题 5: 模 24 计数器	9
1.6 题 6: 模 60 计数器 (二进制)	9
1.7 题 7: 模 10 BCD 计数器	10
1.8 题 8: 模 60 BCD 计数器 (级联方案)	11
1.9 题 9: 模 24 BCD 计数器 (级联方案)	13
1.10 题 10: 模 100 BCD 计数器	14
1.11 题 11: 带使能端 en 的模 8 计数器	15
1.12 题 12: 带同步加载的模 10 计数器	16
1.13 题 13: 双向计数器 (递增/递减)	17
1.14 题 14: 多个计数器级联——模 40	17
1.15 题 15: 给定 VHDL 推断模值	19
1.16 题 16: 带暂停的模 12 计数器	19
1.17 题 17: 倒计时计数器 (递减, 预置值 $\rightarrow 0$)	20
1.18 题 18: 数字钟核心——时/分/秒级联	21
1.19 题 19: 计数器最大工作频率分析	23
1.20 题 20: 格雷码计数器	23
1.21 计数器类题目: 统一秒杀总结	25
第二部分 分频器变式题解析 (15 题)	26
第二章 分频器变式题 (15 题完整解析)	27
2.1 题 1: 2 分频器 (D 触发器自翻转)	27
2.2 题 2: 4 分频器 (用计数器 Q1 输出)	28

目录	3
----	---

2.3 题 3: 8 分频器 (Q2 输出)	28
2.4 题 4: 6 分频器 (占空比 50%)	29
2.5 题 5: 10 分频器	30
2.6 题 6: 3 分频器 (奇数分频, 双沿法)	31
2.7 题 7: 5 分频器	32
2.8 题 8: 100 分频器	33
2.9 题 9: 秒脉冲发生器 (1Hz)	34
2.10 题 10: 可调分频比的分频器	34
2.11 题 11: 占空比可调分频器	35
2.12 题 12: 多级分频器级联	36
2.13 题 13: 给定波形推断分频比	37
2.14 题 14: Q_n 频率计算	37
2.15 题 15: 任意整数分频的统一设计流程	38

第三部分 移位寄存器变式题解析 (15 题) 40

第三章 移位寄存器变式题 (15 题完整解析) 41

3.1 题 1: 串入并出 (SIPO)	41
3.2 题 2: 并串转换 (PISO)	42
3.3 题 3: 双向移位寄存器	43
3.4 题 4: 74LS194 风格通用移位寄存器	44
3.5 题 5: 8 位移位寄存器	45
3.6 题 6: 循环移位寄存器	46
3.7 题 7: 算术右移寄存器	46
3.8 题 8: 移位寄存器实现 $\times 2$ 和 $\div 2$	47
3.9 题 9: 移位寄存器序列检测方案	48
3.10 题 10: 延迟线	49
3.11 题 11: 串行通信接收器	49
3.12 题 12: 给定初值和输入序列, 推断寄存器状态	50
3.13 题 13: 4 位 LFSR (伪随机序列)	51
3.14 题 14: 移位寄存器 Testbench 设计	52
3.15 题 15: 移位寄存器设计范式总结	53

第四部分 环形计数器变式题解析 (15 题) 54

第四章 环形计数器变式题 (15 题完整解析) 55

4.1 题 1: 4 位环形计数器	55
-------------------	----

4.2	题 2: 8 位环形计数器	56
4.3	题 3: 4 位约翰逊计数器 (扭环形)	57
4.4	题 4: 环形计数器 vs 二进制计数器	58
4.5	题 5: 自启动环形计数器	58
4.6	题 6: 自启动约翰逊计数器	59
4.7	题 7: 用环形计数器做序列发生器	60
4.8	题 8: 左移环形计数器	61
4.9	题 9: N 位环形计数器状态数推导	61
4.10	题 10: 5 位环形计数器时序控制器	61
4.11	题 11: 给定波形判断计数器类型	62
4.12	题 12: LFSR vs 环形计数器	63
4.13	题 13: 约翰逊计数器做分频器	63
4.14	题 14: 任意 N 位环形计数器模板	64
4.15	题 15: 环形计数器类统一秒杀	65

第五部分 序列检测器变式题解析 (20 题) 66

第五章	序列检测器变式题 (20 题完整解析)	67
5.1	题 1: 检测 1011 (Moore 型, 非重叠)	67
5.2	题 2: 检测 1011 (重叠检测)	70
5.3	题 3: 检测 1101 (重叠)	70
5.4	题 4: 检测 0101 (重叠)	71
5.5	题 5: 检测 1111 (连续 4 个 1)	72
5.6	题 6: 检测 1001 (重叠)	73
5.7	题 7: 检测 010 (3 位序列)	73
5.8	题 8: 检测 101 (3 位序列)	74
5.9	题 9: 同时检测 1011 和 1101 (合并状态机)	74
5.10	题 10: 检测序列长度可配置 (移位寄存器方案)	76
5.11	题 11: 统计检测次数	76
5.12	题 12: 带超时检测的序列检测器	77
5.13	题 13: Moore 型 vs Mealy 型对比	78
5.14	题 14: 4 位并行序列检测	78
5.15	题 15: 检测包含“无关位”的序列	79
5.16	题 16: 给定状态图写 VHDL	79
5.17	题 17: 给定输入序列和状态机, 求输出序列	80
5.18	题 18: 状态编码优化	81
5.19	题 19: 序列检测器模板化	81

目录	5
5.20 题 20: 序列检测器秒杀总结	83

第一部分

计数器变式题解析（20 题）

第一章 计数器变式题（20 题完整解析）

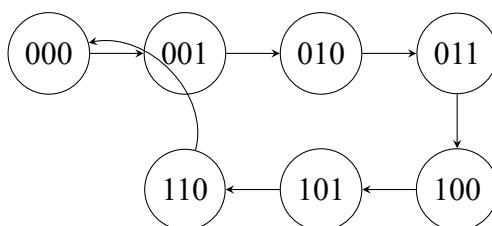
1.1 题 1：模 7 计数器

题目：设计模 7 计数器（ $0 \rightarrow 1 \rightarrow \dots \rightarrow 6 \rightarrow 0 \rightarrow \dots$ ），写 VHDL 和 Testbench。

完整解答

【分析】 $N = 7$ ，位宽 $k = \lceil \log_2 7 \rceil = 3$ 。清零条件： $cnt = 6(110)$ 。

【状态图】



【状态转移表】

当前状态 (十进制)	当前状态 (二进制)	下一状态
0	000	001
1	001	010
2	010	011
3	011	100
4	100	101
5	101	110
6	110	000 (清零)

【VHDL】

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity counter_mod7 is
```

```

6   port (clk, rst_n : in std_logic;
7         count : out std_logic_vector(2 downto 0);
8         carry : out std_logic);
9   end counter_mod7;
10
11  architecture behavioral of counter_mod7 is
12    signal cnt : std_logic_vector(2 downto 0);
13  begin
14    process(clk, rst_n)
15    begin
16      if rst_n = '0' then
17        cnt <= "000";
18      elsif rising_edge(clk) then
19        if cnt = "110" then      -- =6
20          cnt <= "000";
21        else
22          cnt <= cnt + 1;
23        end if;
24      end if;
25    end process;
26    count <= cnt;
27    carry <= '1' when cnt = "110" else '0';
28  end behavioral;

```

【Testbench】

```

1   entity counter_mod7_tb is end entity;
2   architecture test of counter_mod7_tb is
3     signal clk, rst_n, carry : std_logic;
4     signal count : std_logic_vector(2 downto 0);
5     constant T : time := 20 ns;
6   begin
7     uut: entity work.counter_mod7 port map(clk, rst_n, count, carry);
8     clk <= not clk after T/2;
9     process begin
10      rst_n <= '0'; wait for 30 ns;
11      rst_n <= '1';
12      -- 观察两个完整循环(14拍)
13      wait for 14*T;
14      wait;
15    end process;
16  end test;

```

【波形分析】

时钟拍	count(二进制)	十进制
1	001	1
2	010	2
...	...	...
6	110	6
7	000	0 (carry=1!)
8	001	1 (新循环)

验证：第 7 拍 count 归零且 carry=1，第 8 拍 count=1，循环正确。

秒杀口诀

$k = \lceil \log_2 N \rceil$ ，清零条件 $=N-1$ 。套模板：if cnt = N-1 then cnt<=0; else cnt<=cnt+1;

1.2 题 2: 模 5 计数器

题目：设计模 5 计数器 ($0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 0 \rightarrow \dots$)，写出完整 VHDL。

完整解答

【分析】 $N = 5$ ， $k = \lceil \log_2 5 \rceil = 3$ 位。清零条件：cnt = 4(100)。

【状态序列】 000 $\rightarrow$ 001 $\rightarrow$ 010 $\rightarrow$ 011 $\rightarrow$ 100 $\rightarrow$ 000

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity counter_mod5 is
6      port (clk, rst_n : in std_logic;
7            count : out std_logic_vector(2 downto 0);
8            carry : out std_logic);
9  end counter_mod5;
10
11 architecture behavioral of counter_mod5 is
12     signal cnt : std_logic_vector(2 downto 0);
13 begin

```

```

14 process(clk, rst_n)
15 begin
16     if rst_n = '0' then cnt <= "000";
17     elsif rising_edge(clk) then
18         if cnt = "100" then cnt <= "000"; -- =4 清零
19         else cnt <= cnt + 1;
20         end if;
21     end if;
22 end process;
23 count <= cnt;
24 carry <= '1' when cnt = "100" else '0';
25 end behavioral;

```

【验证】5 拍一个循环。第 5 拍 cnt=100(4) 且 carry=1，下一拍归零。

【注意】状态 101/110/111 (5-7) 永远不会出现——这是正常现象，3 位能表示 0-7 但只用了 0-4。

1.3 题 3：模 12 计数器

题目：设计模 12 计数器 (0→1→...→11→0)，并解释为什么不能自然溢出。

完整解答

【分析】 $N = 12$, $k = \lceil \log_2 12 \rceil = 4$ 位。清零条件: $cnt = 11(1011)$ 。

【为什么不能自然溢出】4 位自然溢出 = 模 16。如果不加清零逻辑, $1111+1=0000$, 会经过 16 个状态而不是 12 个。必须显式在 cnt=11 时清零。

【VHDL】

```

1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity counter_mod12 is
6     port (clk, rst_n : in std_logic;
7           count : out std_logic_vector(3 downto 0);
8           carry : out std_logic);
9 end counter_mod12;
10
11 architecture behavioral of counter_mod12 is
12     signal cnt : std_logic_vector(3 downto 0);

```

```

13 begin
14   process(clk, rst_n)
15   begin
16     if rst_n = '0' then cnt <= "0000";
17     elsif rising_edge(clk) then
18       if cnt = "1011" then cnt <= "0000";  -- =11 清零
19       else cnt <= cnt + 1;
20       end if;
21     end if;
22   end process;
23   count <= cnt;
24   carry <= '1' when cnt = "1011" else '0';
25 end behavioral;

```

【Testbench 关键验证点】

- 第 12 拍: cnt=1011(11), carry=1
- 第 13 拍: cnt=0000(0) (归零!)
- 第 14 拍: cnt=0001(1) (新循环开始)

1.4 题 4: 模 13 计数器

题目: 设计模 13 计数器 (0→1→...→12→0)。

完整解答

【分析】 $N = 13$, $k = \lceil \log_2 13 \rceil = 4$ 位。清零条件: $cnt = 12(1100)$ 。

【VHDL】

```

1 signal cnt : std_logic_vector(3 downto 0);
2 -- 清零条件: cnt = "1100" (=12)
3 if cnt = "1100" then cnt <= "0000";
4 else cnt <= cnt + 1; end if;

```

【完整状态序列】

0000 → 0001 → 0010 → 0011 → 0100 → 0101 → 0110 → 0111
 → 1000 → 1001 → 1010 → 1011 → 1100 → 0000

共 13 个状态。状态 1101/1110/1111 (13-15) 永不出现。

1.5 题 5: 模 24 计数器

题目: 设计模 24 计数器 ($0 \rightarrow 1 \rightarrow \dots \rightarrow 23 \rightarrow 0$), 用于小时计数器。

完整解答

【分析】 $N = 24$, $k = \lceil \log_2 24 \rceil = 5$ 位。清零条件: $cnt = 23(10111)$ 。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity counter_mod24 is
6      port (clk, rst_n : in std_logic;
7            count : out std_logic_vector(4 downto 0);
8            carry : out std_logic);
9  end counter_mod24;
10
11 architecture behavioral of counter_mod24 is
12     signal cnt : std_logic_vector(4 downto 0);
13 begin
14     process(clk, rst_n)
15     begin
16         if rst_n = '0' then cnt <= (others => '0');
17         elsif rising_edge(clk) then
18             if cnt = "10111" then cnt <= (others => '0');
19             else cnt <= cnt + 1;
20             end if;
21         end if;
22     end process;
23     count <= cnt;
24     carry <= '1' when cnt = "10111" else '0';
25 end behavioral;

```

【状态范围】 00000(0)→00001(1)→...→10111(23)→00000(0)

注意: 5 位最多表示 31, 剩余 8 个状态 (11000-11111) 永不出现。

【应用】 数字钟的小时计数器。

1.6 题 6: 模 60 计数器 (二进制)

题目: 设计模 60 计数器 ($0 \rightarrow 1 \rightarrow \dots \rightarrow 59 \rightarrow 0$), 用于秒/分计数器。

完整解答

【分析】 $N = 60$, $k = \lceil \log_2 60 \rceil = 6$ 位。清零条件: $cnt = 59(111011)$ 。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity counter_mod60 is
6      port (clk, rst_n : in std_logic;
7            count : out std_logic_vector(5 downto 0);
8            carry : out std_logic);
9  end counter_mod60;
10
11 architecture behavioral of counter_mod60 is
12     signal cnt : std_logic_vector(5 downto 0);
13 begin
14     process(clk, rst_n)
15     begin
16         if rst_n = '0' then cnt <= (others => '0');
17         elsif rising_edge(clk) then
18             if cnt = "111011" then cnt <= (others => '0');
19             else cnt <= cnt + 1;
20             end if;
21         end if;
22     end process;
23     count <= cnt;
24     carry <= '1' when cnt = "111011" else '0';
25 end behavioral;

```

【二进制 vs BCD 模 60】

- 二进制模 60: 一个 6 位整体计数器 (如本题)
- BCD 模 60: 十位 (模 6)+ 个位 (模 10) 级联
- 考试中要注意区分”二进制模 60”和”BCD 模 60”

1.7 题 7: 模 10 BCD 计数器

题目: 设计模 10 BCD 计数器 ($0 \rightarrow 1 \rightarrow \dots \rightarrow 9 \rightarrow 0$)。个位, 4 位输出, BCD 编码。

完整解答

【分析】 $N = 10$, $k = 4$ 位。状态: 0000→0001→...→1001→0000。状态 1010-1111 不允许出现。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity bcd_counter_mod10 is
6      port (clk, rst_n : in std_logic;
7            bcd : out std_logic_vector(3 downto 0);
8            carry : out std_logic);
9  end bcd_counter_mod10;
10
11 architecture behavioral of bcd_counter_mod10 is
12     signal cnt : std_logic_vector(3 downto 0);
13 begin
14     process(clk, rst_n)
15     begin
16         if rst_n = '0' then cnt <= "0000";
17         elsif rising_edge(clk) then
18             if cnt = "1001" then cnt <= "0000"; -- =9 清零
19             else cnt <= cnt + 1;
20             end if;
21         end if;
22     end process;
23     bcd <= cnt;
24     carry <= '1' when cnt = "1001" else '0';
25 end behavioral;

```

【关键】 与普通模 10 计数器的区别: BCD 的每个十进制位是独立 4 位, 显示友好。

1.8 题 8: 模 60 BCD 计数器（级联方案）

题目: 设计 BCD 模 60 计数器, 个位 (模 10)+ 十位 (模 6)。输出十位和个位各 4 位 BCD 码。

完整解答

【分析】个位：模 10，满 9→清零并向十位进位。十位：收到进位 +1，十位模 6 (0-5)。整体清零条件：十位 =5 且个位 =9 (即 59)。

【VHDL】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.std_logic_unsigned.all;
4
5  entity bcd_mod60 is
6      port (clk, rst_n : in std_logic;
7            tens, units : out std_logic_vector(3 downto 0));
8  end bcd_mod60;
9
10 architecture behavioral of bcd_mod60 is
11     signal u_cnt : std_logic_vector(3 downto 0); -- 个位
12     signal t_cnt : std_logic_vector(3 downto 0); -- 十位
13 begin
14     process(clk, rst_n)
15     begin
16         if rst_n = '0' then
17             u_cnt <= "0000"; t_cnt <= "0000";
18         elsif rising_edge(clk) then
19             -- 整体清零：59→00
20             if t_cnt = "0101" and u_cnt = "1001" then
21                 u_cnt <= "0000"; t_cnt <= "0000";
22             -- 个位满9→进位
23             elsif u_cnt = "1001" then
24                 u_cnt <= "0000";
25                 t_cnt <= t_cnt + 1;
26             else
27                 u_cnt <= u_cnt + 1;
28             end if;
29         end if;
30     end process;
31     tens <= t_cnt; units <= u_cnt;
32 end behavioral;

```

【状态演化 (关键节点)】

拍	十位 (tens)	个位 (units)
9	0000	1001
10	0001	0000 (个位进位!)
19	0001	1001
20	0010	0000
...	...	...
59	0101	1001
60	0000	0000 (整体清零!)

【验证要点】第 10 拍：个位从 9 变 0，十位从 0 变 1——进位正确。第 60 拍：59→00——整体清零正确。

1.9 题 9：模 24 BCD 计数器（级联方案）

题目：设计 BCD 模 24 计数器。十位模 3（0-2），个位模 10（0-9）。整体清零条件：23。

完整解答

【VHDL】

```

1 entity bcd_mod24 is
2     port (clk, rst_n : in std_logic;
3           tens, units : out std_logic_vector(3 downto 0));
4 end bcd_mod24;
5
6 architecture behavioral of bcd_mod24 is
7     signal u_cnt, t_cnt : std_logic_vector(3 downto 0);
8 begin
9     process(clk, rst_n)
10    begin
11        if rst_n = '0' then
12            u_cnt <= "0000"; t_cnt <= "0000";
13        elsif rising_edge(clk) then
14            -- 整体清零：23→00
15            if t_cnt = "0010" and u_cnt = "0011" then
16                u_cnt <= "0000"; t_cnt <= "0000";
17            elsif u_cnt = "1001" then
18                u_cnt <= "0000";

```



```

19         t_cnt <= t_cnt + 1;
20     else
21         u_cnt <= u_cnt + 1;
22     end if;
23 end if;
24 end process;
25 tens <= t_cnt; units <= u_cnt;
26 end behavioral;

```

【注意】十位的模 3 约束由清零条件隐式保证——计到 23 就清零，所以十位永远不会达到 3。

1.10 题 10: 模 100 BCD 计数器

题目：设计 BCD 模 100 计数器（00→01→...→99→00）。

完整解答

【分析】个位模 10，十位模 10。清零条件：十位 = 9 且个位 = 9（即 99）。

【VHDL】

```

1  entity bcd_mod100 is
2      port (clk, rst_n : in std_logic;
3            tens, units : out std_logic_vector(3 downto 0));
4  end bcd_mod100;
5
6  architecture behavioral of bcd_mod100 is
7      signal u_cnt, t_cnt : std_logic_vector(3 downto 0);
8  begin
9      process(clk, rst_n)
10     begin
11         if rst_n = '0' then
12             u_cnt <= "0000"; t_cnt <= "0000";
13         elsif rising_edge(clk) then
14             if t_cnt = "1001" and u_cnt = "1001" then -- 99
15                 u_cnt <= "0000"; t_cnt <= "0000";
16             elsif u_cnt = "1001" then
17                 u_cnt <= "0000";
18                 t_cnt <= t_cnt + 1;
19             else
20                 u_cnt <= u_cnt + 1;

```

```

21     end if;
22     end if;
23 end process;
24 tens <= t_cnt; units <= u_cnt;
25 end behavioral;

```

1.11 题 11：带使能端 en 的模 8 计数器

题目：设计带使能信号的模 8 计数器。en=1 计数，en=0 保持当前值。

完整解答

【分析】模 $8=2^3$ ，自然溢出即可。增加 en 控制是否 +1。

【VHDL】

```

1  entity counter_mod8_en is
2      port (clk, rst_n, en : in std_logic;
3            count : out std_logic_vector(2 downto 0));
4  end counter_mod8_en;
5
6  architecture behavioral of counter_mod8_en is
7      signal cnt : std_logic_vector(2 downto 0);
8  begin
9      process(clk, rst_n)
10     begin
11         if rst_n = '0' then cnt <= "000";
12         elsif rising_edge(clk) then
13             if en = '1' then
14                 cnt <= cnt + 1; -- 3位自然溢出=模8
15             end if;
16             -- en='0': 不赋值 → 保持
17         end if;
18     end process;
19     count <= cnt;
20 end behavioral;

```

【Testbench 关键测试】

- en=1 时正常计数 (0,1,2,...→7→0)
- en=0 时 count 保持不变

- en 从 0 切换为 1 时从当前值继续计数

1.12 题 12：带同步加载的模 10 计数器

题目：设计带 load 端口的模 10 计数器。load=1 时 cnt=data_in；load=0 时正常计数 (0-9)。

完整解答

【VHDL】

```
1 entity counter_mod10_load is
2   port (clk, rst_n, load : in std_logic;
3         data_in : in std_logic_vector(3 downto 0);
4         count : out std_logic_vector(3 downto 0));
5 end counter_mod10_load;
6
7 architecture behavioral of counter_mod10_load is
8   signal cnt : std_logic_vector(3 downto 0);
9 begin
10  process(clk, rst_n)
11  begin
12    if rst_n = '0' then cnt <= "0000";
13    elsif rising_edge(clk) then
14      if load = '1' then
15        cnt <= data_in;          -- 同步加载
16      elsif cnt = "1001" then
17        cnt <= "0000";          -- 满9清零
18      else
19        cnt <= cnt + 1;
20      end if;
21    end if;
22  end process;
23  count <= cnt;
24 end behavioral;
```

【注意】load 优先级高于计数。if-else 顺序决定优先级——if 在最前面优先级最高。

1.13 题 13：双向计数器（递增/递减）

题目：设计模 16 双向计数器。up=1 递增，up=0 递减。

完整解答

【VHDL】

```
1 entity counter_updown is
2     port (clk, rst_n, up : in std_logic;
3           count : out std_logic_vector(3 downto 0));
4 end counter_updown;
5
6 architecture behavioral of counter_updown is
7     signal cnt : std_logic_vector(3 downto 0);
8 begin
9     process(clk, rst_n)
10    begin
11        if rst_n = '0' then cnt <= "0000";
12        elsif rising_edge(clk) then
13            if up = '1' then
14                cnt <= cnt + 1; -- 递增
15            else
16                cnt <= cnt - 1; -- 递减
17            end if;
18        end if;
19    end process;
20    count <= cnt;
21 end behavioral;
```

【递减行为】cnt-1：当 cnt=0000 时，0000-1=1111（二进制借位），所以是模 16 递减计数器。

【状态序列（递减）】0000 → 1111 → 1110 → … → 0001 → 0000

1.14 题 14：多个计数器级联——模 40

题目：模 8 计数器 + 模 5 计数器级联。总模值？写出联逻辑。

完整解答

【分析】总模值 $= M_1 \times M_2 = 8 \times 5 = 40$ 。

【级联逻辑】模 8 计数器每 8 拍 carry=1 一次。模 5 计数器以 carry 作为使能，收到 carry=1 时 +1。

【VHDL】

```

1 entity counter_mod40_cascade is
2   port (clk, rst_n : in std_logic;
3         count_lo : out std_logic_vector(2 downto 0); -- 模 8
4         count_hi : out std_logic_vector(2 downto 0)); -- 模 5
5 end counter_mod40_cascade;
6
7 architecture behavioral of counter_mod40_cascade is
8   signal cnt8 : std_logic_vector(2 downto 0);
9   signal cnt5 : std_logic_vector(2 downto 0);
10  signal carry8 : std_logic;
11 begin
12  -- 模 8 (低位)
13  process(clk, rst_n)
14  begin
15    if rst_n = '0' then cnt8 <= "000";
16    elsif rising_edge(clk) then
17      cnt8 <= cnt8 + 1; -- 3位自然溢出=模 8
18    end if;
19  end process;
20  carry8 <= '1' when cnt8 = "111" else '0';
21
22  -- 模 5 (高位)
23  process(clk, rst_n)
24  begin
25    if rst_n = '0' then cnt5 <= "000";
26    elsif rising_edge(clk) then
27      if carry8 = '1' then
28        if cnt5 = "100" then cnt5 <= "000"; -- =4 清零
29        else cnt5 <= cnt5 + 1;
30      end if;
31    end if;
32  end process;
33
34  count_lo <= cnt8; count_hi <= cnt5;
35 end behavioral;
36

```

【通用公式】级联总模值 $= M_1 \times M_2 \times \cdots \times M_k$

1.15 题 15：给定 VHDL 推断模值

题目：分析以下代码，推断计数器的模值 N。

```
1  if cnt = "1010" then cnt <= "0000";  
2  else cnt <= cnt + 1; end if;
```

完整解答

【分析】清零条件：cnt=1010=10。

【关键公式】模值 = 清零比较值 + 1

当 cnt=10 时清零，意味着 cnt 经过了状态 0,1,2,...,10，共 11 个状态后被清零。

所以 $N=10+1=11$ 。

【验证】状态序列：0→1→2→3→4→5→6→7→8→9→10→0 共 11 个不同的状态（0 到 10），模值 = 11。

【常见陷阱】不能只看清零值！清零值 = 10 但模值 = 11。因为 cnt=10 这一个状态本身也算一个状态。

【万能公式】 $N = (\text{清零比较的数值}) + 1$

秒杀口诀

模值 = 清零条件中的比较值 + 1

cnt = X 时清零 → 模值 = X + 1

1.16 题 16：带暂停的模 12 计数器

题目：设计带暂停信号 pause 的模 12 计数器。pause=1 暂停，pause=0 恢复。

完整解答

【VHDL】

```
1  entity counter_mod12_pause is  
2      port (clk, rst_n, pause : in std_logic;  
3            count : out std_logic_vector(3 downto 0));  
4  end counter_mod12_pause;  
5  
6  architecture behavioral of counter_mod12_pause is
```

```

7  signal cnt : std_logic_vector(3 downto 0);
8  begin
9  process(clk, rst_n)
10 begin
11     if rst_n = '0' then cnt <= "0000";
12     elsif rising_edge(clk) then
13         if pause = '0' then
14             if cnt = "1011" then cnt <= "0000";
15             else cnt <= cnt + 1;
16             end if;
17         end if;
18         -- pause='1': 不赋值  $\rightarrow$  保持
19     end if;
20 end process;
21 count <= cnt;
22 end behavioral;

```

【Testbench 关键验证】

- pause=0: 正常 $0 \rightarrow 1 \rightarrow \dots \rightarrow 11 \rightarrow 0$
- pause=1: cnt 保持不变
- pause 从 $1 \rightarrow 0$: 从保持的值继续计数

1.17 题 17: 倒计时计数器 (递减, 预置值 $\rightarrow 0$)

题目: 设计从预置值 PRE 递减到 0 的倒计时计数器。到 0 后重新加载 PRE。

完整解答**【VHDL】**

```

1  entity down_counter is
2  generic (PRE : integer := 9);
3  port (clk, rst_n : in std_logic;
4         count : out std_logic_vector(3 downto 0);
5         zero : out std_logic);
6  end down_counter;
7
8  architecture behavioral of down_counter is

```

```

9   signal cnt : std_logic_vector(3 downto 0);
10  begin
11    process(clk, rst_n)
12    begin
13      if rst_n = '0' then
14        cnt <= std_logic_vector(to_unsigned(PRE, 4));
15      elsif rising_edge(clk) then
16        if cnt = "0000" then
17          cnt <= std_logic_vector(to_unsigned(PRE, 4));
18        else
19          cnt <= cnt - 1;
20        end if;
21      end if;
22    end process;
23    count <= cnt;
24    zero <= '1' when cnt = "0000" else '0';
25  end behavioral;

```

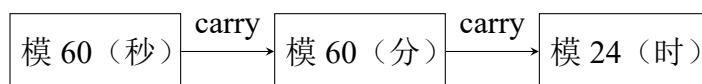
【状态序列 (PRE=9)】 $9 \rightarrow 8 \rightarrow 7 \rightarrow 6 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 2 \rightarrow 1 \rightarrow 0 \rightarrow 9 \rightarrow 8 \rightarrow \dots$

1.18 题 18: 数字钟核心——时/分/秒级联

题目: 设计时 (H): 分 (M): 秒 (S) 三层级联 BCD 计数器。S 和 M 为模 60, H 为模 24。

完整解答

【架构】



秒满 60 → 分 +1, 分满 60 → 时 +1, 时满 24 → 时归零。

【VHDL】

```

1  entity digital_clock is
2    port (clk, rst_n : in std_logic;
3          sec_u, sec_t : out std_logic_vector(3 downto 0);
4          min_u, min_t : out std_logic_vector(3 downto 0);
5          hour_u, hour_t : out std_logic_vector(3 downto 0));
6  end digital_clock;

```



```

7
8 architecture behavioral of digital_clock is
9     signal s_u, s_t, m_u, m_t, h_u, h_t : std_logic_vector(3 downto 0);
10    signal s_carry, m_carry : std_logic;
11 begin
12     process(clk, rst_n)
13     begin
14         if rst_n = '0' then
15             s_u <= x"0"; s_t <= x"0"; m_u <= x"0"; m_t <= x"0";
16             h_u <= x"0"; h_t <= x"0"; s_carry <= '0'; m_carry <= '0';
17         elsif rising_edge(clk) then
18             s_carry <= '0'; m_carry <= '0';
19
20             -- 秒个位计数
21             if s_u = x"9" then s_u <= x"0";
22                 if s_t = x"5" then
23                     s_t <= x"0"; s_carry <= '1'; -- 59→00秒, 进位到分
24                 else s_t <= s_t + 1;
25                 end if;
26             else s_u <= s_u + 1;
27             end if;
28
29             -- 分计数 (收到秒进位时)
30             if s_carry = '1' then
31                 if m_u = x"9" then m_u <= x"0";
32                     if m_t = x"5" then
33                         m_t <= x"0"; m_carry <= '1'; -- 59→00分, 进位到时
34                     else m_t <= m_t + 1;
35                     end if;
36                 else m_u <= m_u + 1;
37                 end if;
38             end if;
39
40             -- 时计数 (收到分进位时)
41             if m_carry = '1' then
42                 if h_t = x"2" and h_u = x"3" then -- 23→00
43                     h_t <= x"0"; h_u <= x"0";
44                 elsif h_u = x"9" then
45                     h_u <= x"0"; h_t <= h_t + 1;
46                 else h_u <= h_u + 1;
47                 end if;
48             end if;
49         end if;

```

```

50  end process;
51
52  sec_u <= s_u; sec_t <= s_t;
53  min_u <= m_u; min_t <= m_t;
54  hour_u <= h_u; hour_t <= h_t;
55  end behavioral;

```

1.19 题 19：计数器最大工作频率分析

题目：4 位行波进位计数器，D 触发器 $t_{cko} = 2\text{ns}$ ， $t_{setup} = 1\text{ns}$ ，每级进位延迟 3ns 。求 f_{max} 。

完整解答

【关键路径】从 D 触发器输出 → 加法器进位链（4 级 $\times 3\text{ns} = 12\text{ns}$ ）→ 下一个 D 触发器 setup 时间

$$\begin{aligned}
 t_{critical} &= t_{cko} + 4 \times t_{carry} + t_{setup} \\
 &= 2 + 12 + 1 = 15\text{ns}
 \end{aligned}$$

$$f_{max} = \frac{1}{15\text{ns}} = 66.7\text{MHz}$$

【如果改用超前进位】进位延迟从 $O(N)$ 降为 $O(1)$ ， f_{max} 可大幅提升。

【考试公式】 $f_{max} = 1/(t_{cko} + N \times t_{carry} + t_{setup})$

N = 计数器位数， t_{carry} = 每级进位延迟。

1.20 题 20：格雷码计数器

题目：设计 3 位格雷码计数器（000→001→011→010→110→111→101→100→000...）。

完整解答

【分析】格雷码：相邻状态只有 1 位不同。3 位格雷码共 8 个状态（模 8）。

【格雷码序列（3 位）】

状态编号	格雷码	下一状态格雷码
0	000	001
1	001	011
2	011	010
3	010	110
4	110	111
5	111	101
6	101	100
7	100	000

【VHDL——直接 case 实现】

```

1  entity gray_counter_3bit is
2      port (clk, rst_n : in std_logic;
3            gray : out std_logic_vector(2 downto 0));
4  end gray_counter_3bit;
5
6  architecture behavioral of gray_counter_3bit is
7      signal g : std_logic_vector(2 downto 0);
8  begin
9      process(clk, rst_n)
10     begin
11         if rst_n = '0' then g <= "000";
12         elsif rising_edge(clk) then
13             case g is
14                 when "000" => g <= "001";
15                 when "001" => g <= "011";
16                 when "011" => g <= "010";
17                 when "010" => g <= "110";
18                 when "110" => g <= "111";
19                 when "111" => g <= "101";
20                 when "101" => g <= "100";
21                 when "100" => g <= "000";
22                 when others => g <= "000";
23             end case;
24         end if;
25     end process;
26     gray <= g;
27 end behavioral;

```

【应用】 格雷码计数器常用于 FIFO 的读写指针——相邻状态只变 1 位，减少亚稳态风险。

1.21 计数器类题目：统一秒杀总结

计数器 20 题秒杀总结

步骤 1：确定模值 N ，位宽 $k = \lceil \log_2 N \rceil$

步骤 2：清零条件 = $N-1$ （二进制）

步骤 3：套用统一模板：

```
if cnt = (N-1) then cnt <= 0; else cnt <= cnt + 1; end if;
```

步骤 4：如有额外控制（en, load, pause, up/down），用 if-else 设定优先级

步骤 5：BCD 级联：低位 carry 接高位使能，整体清零条件同时清零多级

关键公式：

- 模值 = 清零比较值 + 1
- 级联总模值 = $M_1 \times M_2 \times \dots$
- 2^N 模值（如模 16、模 8）：不需要显式清零（自然溢出）
- $f_{max} = 1 / (t_{cko} + N \times t_{carry} + t_{setup})$

所有计数器题目的答案都是同一个模板改参数。

第二部分

分频器变式题解析（15 题）

第二章 分频器变式题（15 题完整解析）

2.1 题 1：2 分频器（D 触发器自翻转）

题目：用 D 触发器实现 2 分频器。 $f_{out} = f_{clk}/2$ 。

完整解答

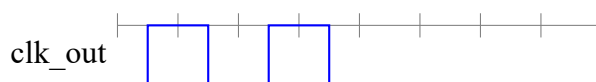
【分析】 一个 D 触发器， $\overline{Q}$ 接回 D。每个时钟上升沿 Q 翻转一次。Q 的周期 = 2 个时钟周期 → 二分频。

【原理】 $D = \overline{Q} \rightarrow Q(n+1) = \overline{Q(n)} \rightarrow$ 每拍翻转。

【VHDL】

```
1 entity div2 is
2     port (clk, rst_n : in std_logic; clk_out : out std_logic);
3 end div2;
4
5 architecture behavioral of div2 is
6     signal q : std_logic;
7 begin
8     process(clk, rst_n)
9     begin
10         if rst_n = '0' then q <= '0';
11         elsif rising_edge(clk) then q <= not q; end if;
12     end process;
13     clk_out <= q;
14 end behavioral;
```

【波形】



周期 = 2T → 频率 = $f_{clk}/2$

【验证】 $Q(t)=0,1,0,1,\dots$ — 每 2 拍完成一个 $0 \rightarrow 1 \rightarrow 0$ 循环。

2.2 题 2：4 分频器（用计数器 Q1 输出）

题目：用 2 位计数器，取 Q1 输出实现 4 分频。

完整解答

【分析】Q1 每 4 拍完成一次 0→1→0 循环。频率 = $f_{clk}/4$ 。

【VHDL】

```

1  entity div4 is
2      port (clk, rst_n : in std_logic; clk_out : out std_logic);
3  end div4;
4
5  architecture behavioral of div4 is
6      signal cnt : std_logic_vector(1 downto 0);
7  begin
8      process(clk, rst_n)
9      begin
10         if rst_n = '0' then cnt <= "00";
11         elsif rising_edge(clk) then cnt <= cnt + 1; end if;
12     end process;
13     clk_out <= cnt(1);  -- Q1天然4分频!
14 end behavioral;

```

【逐拍验证 Q1】

拍	cnt	Q1	说明
1	01	0	
2	10	1	Q1 翻转
3	11	1	
4	00	0	Q1 翻转

Q1: 0→0→1→1→0→0→... 一每 4 拍一个周期。

2.3 题 3：8 分频器（Q2 输出）

题目：用 3 位计数器实现 8 分频。

完整解答

【VHDL】

```
1 signal cnt : std_logic_vector(2 downto 0);
2 -- cnt(2) = Q2, 天然8分频
3 clk_out <= cnt(2);
```

【通用公式】 $f_{Q_k} = f_{clk}/2^{k+1}$

输出位	分频比
Q0	2 分频
Q1	4 分频
Q2	8 分频
Q3	16 分频
Qk	2^{k+1} 分频

2.4 题 4：6 分频器（占空比 50%）

题目：设计 6 分频器，占空比 50%（3 拍高 +3 拍低）。

完整解答

【分析】模 6 计数器（0-5）。cnt<3 输出 0，cnt≥3 输出 1。

【VHDL】

```
1 entity div6 is
2   port (clk, rst_n : in std_logic; clk_out : out std_logic);
3 end div6;
4
5 architecture behavioral of div6 is
6   signal cnt : std_logic_vector(2 downto 0);
7 begin
8   process(clk, rst_n)
9   begin
10    if rst_n = '0' then cnt <= "000";
11    elsif rising_edge(clk) then
12      if cnt = "101" then cnt <= "000"; -- 模6
13      else cnt <= cnt + 1;
14      end if;
15    end if;
```



```

16  end process;
17  clk_out <= '1' when cnt >= "011" else '0'; -- 0-2低, 3-5高
18  end behavioral;

```

【逐拍验证】

拍	cnt	clk_out
1	000	0
2	001	0
3	010	0
4	011	1
5	100	1
6	101	1
7	000	0 (新周期)

3 拍低 + 3 拍高 = 6 拍一个周期, 占空比恰好 50%。

2.5 题 5: 10 分频器

题目: 设计 10 分频器, 占空比 50%。

完整解答

【VHDL】

```

1  entity div10 is
2    port (clk, rst_n : in std_logic; clk_out : out std_logic);
3  end div10;
4
5  architecture behavioral of div10 is
6    signal cnt : std_logic_vector(3 downto 0);
7  begin
8    process(clk, rst_n)
9    begin
10     if rst_n = '0' then cnt <= "0000";
11     elsif rising_edge(clk) then
12       if cnt = "1001" then cnt <= "0000";
13       else cnt <= cnt + 1;
14       end if;
15     end if;

```

```

16   end process;
17   clk_out <= '1' when cnt >= "0101" else '0'; -- 0-4 低, 5-9 高
18 end behavioral;

```

【偶数分频通用公式】 $\text{clk_out} = 1 \text{ when } cnt \geq M/2 \text{ else } 0$

2.6 题 6: 3 分频器 (奇数分频, 双沿法)

题目: 设计占空比接近 50% 的 3 分频器。

完整解答

【分析】 偶数 $M/2$ 阈值法对奇数 M 不会得到 50% 占空比。解法: 用上升沿 + 下降沿分别产生两个中间信号, 然后 OR 组合。

【VHDL】

```

1  entity div3 is
2      port (clk, rst_n : in std_logic; clk_out : out std_logic);
3  end div3;
4
5  architecture behavioral of div3 is
6      signal cnt : std_logic_vector(1 downto 0);
7      signal pos_out, neg_out : std_logic;
8  begin
9      -- 上升沿计数器
10     process(clk, rst_n)
11     begin
12         if rst_n = '0' then cnt <= "00";
13         elsif rising_edge(clk) then
14             if cnt = "10" then cnt <= "00"; else cnt <= cnt + 1; end if;
15         end if;
16     end process;
17
18     -- 上升沿产生信号
19     process(clk) begin
20         if rising_edge(clk) then
21             if cnt = 0 then pos_out <= '1'; else pos_out <= '0'; end if;
22         end if;
23     end process;
24
25     -- 下降沿产生信号 (比上升沿延迟半个周期)

```

```

26 process(clk) begin
27     if falling_edge(clk) then
28         neg_out <= pos_out;
29     end if;
30 end process;
31
32 clk_out <= pos_out or neg_out;
33 end behavioral;

```

【波形分析】 pos_out: 1-0-0-1-0-0-... (1/3 占空比) neg_out: 0-1-0-0-1-0-... (延迟半拍) OR 结果: 1-1-0-1-1-0-... (接近 50% 占空比! 但实际是 2 高 1 低)

【奇数分频的占空比】严格来说奇数分频无法实现精确 50% 占空比 (因为奇数个时钟周期不能等分), 但双沿法可以做到接近 50%。

2.7 题 7: 5 分频器

题目: 设计 5 分频器 (双沿法实现 50% 占空比)。

完整解答

【VHDL——同 3 分频方案, 仅改模值】

```

1 entity div5 is
2     port (clk, rst_n : in std_logic; clk_out : out std_logic);
3 end div5;
4
5 architecture behavioral of div5 is
6     signal cnt : std_logic_vector(2 downto 0); -- 3位, 模5
7     signal pos_out, neg_out : std_logic;
8 begin
9     process(clk, rst_n)
10    begin
11        if rst_n = '0' then cnt <= "000";
12        elsif rising_edge(clk) then
13            if cnt = "100" then cnt <= "000"; else cnt <= cnt + 1; end if;
14        end if;
15    end process;
16
17    process(clk) begin
18        if rising_edge(clk) then
19            if cnt < 2 then pos_out <= '1'; else pos_out <= '0'; end if;

```

```
20     end if;
21 end process;
22
23 process(clk) begin
24     if falling_edge(clk) then neg_out <= pos_out; end if;
25 end process;
26
27 clk_out <= pos_out or neg_out;
28 end behavioral;
```

2.8 题 8：100 分频器

题目：50MHz 时钟 → 500kHz 信号。求分频比 M，写出 VHDL。

完整解答

【分析】 $M = f_{in}/f_{out} = 50\text{MHz}/500\text{kHz} = 100$ 。偶数分频，使用 M/2 阈值法。

【VHDL】

```
1 entity div100 is
2     port (clk, rst_n : in std_logic; clk_out : out std_logic);
3 end div100;
4
5 architecture behavioral of div100 is
6     signal cnt : integer range 0 to 99;
7 begin
8     process(clk, rst_n)
9     begin
10         if rst_n = '0' then cnt <= 0;
11         elsif rising_edge(clk) then
12             if cnt = 99 then cnt <= 0; else cnt <= cnt + 1; end if;
13         end if;
14     end process;
15     clk_out <= '1' when cnt >= 50 else '0'; -- M/2 = 50
16 end behavioral;
```

【位宽优化】 cnt 用 integer 不需要手动计算位宽，综合器自动优化。也可用 7 位 std_logic_vector ($2^7 = 128 > 99$)。

2.9 题 9: 秒脉冲发生器 (1Hz)

题目: 100MHz 时钟 $\rightarrow$ 1Hz 秒脉冲。 $M = 100,000,000$ 。

完整解答

【分析】 $M=100,000,000$ 。需要 $\lceil \log_2 10^8 \rceil = 27$ 位计数器。

【VHDL】

```

1  entity sec_pulse is
2      port (clk_100MHz, rst_n : in std_logic; pulse_1Hz : out std_logic);
3  end sec_pulse;
4
5  architecture behavioral of sec_pulse is
6      constant M : integer := 100000000;
7      signal cnt : integer range 0 to M-1;
8  begin
9      process(clk_100MHz, rst_n)
10     begin
11         if rst_n = '0' then cnt <= 0;
12         elsif rising_edge(clk_100MHz) then
13             if cnt = M-1 then cnt <= 0; else cnt <= cnt + 1; end if;
14         end if;
15     end process;
16     pulse_1Hz <= '1' when cnt < M/2 else '0';
17     -- 50% 占空比, 周期1秒
18 end behavioral;

```

【注意】 100M 用 integer 没问题——integer 在 VHDL 中是 32 位有符号数, 范围足够。

2.10 题 10: 可调分频比的分频器

题目: 分频比 DIV 可动态设置 (1-16)。

完整解答

【VHDL】

```

1  entity var_div is
2      port (clk, rst_n : in std_logic;

```

```
3      DIV : in std_logic_vector(3 downto 0); -- 分频比
4      clk_out : out std_logic);
5  end var_div;
6
7  architecture behavioral of var_div is
8      signal cnt : std_logic_vector(3 downto 0);
9  begin
10     process(clk, rst_n)
11     begin
12         if rst_n = '0' then cnt <= "0000";
13         elsif rising_edge(clk) then
14             if cnt = DIV - 1 then cnt <= "0000";
15             else cnt <= cnt + 1;
16             end if;
17         end if;
18     end process;
19     clk_out <= '1' when cnt < ('0' & DIV(3 downto 1)) else '0';
20     -- DIV/2作为阈值（50%占空比）
21 end behavioral;
```

2.11 题 11：占空比可调分频器

题目：M=10 分频器，高电平持续拍数 DUTY 可配置。

完整解答

【VHDL】

```
1  entity duty_div is
2      port (clk, rst_n : in std_logic;
3            DUTY : in std_logic_vector(3 downto 0); -- 高电平拍数
4            clk_out : out std_logic);
5  end duty_div;
6
7  architecture behavioral of duty_div is
8      signal cnt : std_logic_vector(3 downto 0);
9  begin
10     process(clk, rst_n)
11     begin
12         if rst_n = '0' then cnt <= "0000";
```

```

13     elsif rising_edge(clk) then
14         if cnt = "1001" then cnt <= "0000"; -- M=10
15         else cnt <= cnt + 1;
16         end if;
17     end if;
18 end process;
19 clk_out <= '1' when cnt < DUTY else '0';
20 -- DUTY=5 → 50\% 占空比, DUTY=2 → 20\% 占空比
21 end behavioral;

```

【考试考点】 改变占空比 = 改变 cnt 的比较阈值。占空比 = DUTY/M。

2.12 题 12：多级分频器级联

题目：先 2 分频再 5 分频 → 总 10 分频。写出完整 VHDL，比较与直接 10 分频的区别。

完整解答

【VHDL】

```

1  entity cascade_div is
2      port (clk, rst_n : in std_logic; clk_out : out std_logic);
3  end cascade_div;
4
5  architecture behavioral of cascade_div is
6      signal div2_out, div5_out : std_logic;
7      signal cnt5 : std_logic_vector(2 downto 0);
8  begin
9      -- 第一级：2分频
10     process(clk, rst_n)
11     begin
12         if rst_n = '0' then div2_out <= '0';
13         elsif rising_edge(clk) then div2_out <= not div2_out; end if;
14     end process;
15
16     -- 第二级：5分频（以div2_out为时钟）
17     process(div2_out, rst_n)
18     begin
19         if rst_n = '0' then cnt5 <= "000";
20         elsif rising_edge(div2_out) then

```

```

21     if cnt5 = "100" then cnt5 <= "000"; else cnt5 <= cnt + 1; end if;
22     end if;
23 end process;
24 div5_out <= '1' when cnt5 >= 2 else '0';
25 clk_out <= div5_out;
26 end behavioral;

```

【总 $M = M_1 \times M_2 = 2 \times 5 = 10$ 】

【两种方案比较】

- 级联：占用更少资源（级联比分频比小，每级计数器更小）
- 直接模 10：结构简单，单一时钟域
- 级联的缺点：后级使用前级输出作为时钟，增加时钟偏斜风险（FPGA 中一般不建议用逻辑输出做时钟）

2.13 题 13：给定波形推断分频比

题目：输出信号每 12 个输入时钟完成一个周期（6 高 6 低）。求 M 。

完整解答

【分析】输出周期 = 12 个输入时钟周期。

$$M = \frac{T_{out}}{T_{clk}} = \frac{12}{1} = 12$$

【答】分频比 $M=12$ （12 分频器）。

【快速判断法】数输出信号的周期包含几个输入时钟周期 → 就是几分频。

2.14 题 14：Qn 频率计算

题目： $f_{clk} = 100\text{MHz}$ ，4 位计数器。求 $Q0, Q1, Q2, Q3$ 频率。

完整解答

输出	分频比	频率
Q0	2	50.0 MHz
Q1	4	25.0 MHz
Q2	8	12.5 MHz
Q3	16	6.25 MHz

【公式】 $f_{Q_k} = f_{clk}/2^{k+1}$

【直观理解】Q0 每拍翻转 → 2 分频。Q1 每 2 拍翻转 → 4 分频。Qk 每 2^k 拍翻转 → 2^{k+1} 分频。

2.15 题 15：任意整数分频的统一设计流程

题目：总结任意整数分频 M 的统一设计流程。

完整解答

【统一流程】

步骤 1：计算分频比 M

$$M = f_{in}/f_{out}$$

步骤 2：判断 M 的奇偶性

- M 为偶数 → 方案 A（简单）
- M 为奇数 → 方案 B（双沿法）

步骤 3（方案 A：偶数分频）

- 设计模 M 计数器
- $\text{clk_out} = '1'$ when $\text{cnt} \geq M/2$ else $'0'$
- 完美 50% 占空比

步骤 3（方案 B：奇数分频）

- 上升沿产生 pos_out （占空比尽量接近 50%）
- 下降沿采样 pos_out 得到 neg_out （延迟半拍）
- $\text{clk_out} = \text{pos_out} \text{ OR } \text{neg_out}$

步骤 4: 特殊情况

- $M = 2^k$: 直接用计数器 Q_{k-1} 输出 (最简单!)
- 级联: $M = M_1 \times M_2$, 多级串联

【VHDL 模板 (偶数分频)】

```
1  if cnt = M-1 then cnt <= 0; else cnt <= cnt + 1; end if;  
2  clk_out <= '1' when cnt >= M/2 else '0';
```

分频器秒杀总结

1. 2^N 分频: 直接取 $Q_{(N-1)}$, 什么额外逻辑都不需要
2. 偶数分频: 模 M 计数器, $M/2$ 阈值决定占空比
3. 奇数分频: 双沿法 (上升沿 + 下降沿 +OR)
4. 级联总分频比: $M = M_1 \times M_2$
5. 占空比 = 高电平拍数 / M

第三部分

移位寄存器变式题解析（15 题）

第三章 移位寄存器变式题（15 题完整解析）

3.1 题 1：串入并出（SIPO）

题目：设计 4 位串入并出移位寄存器。输入 si，输出 po(3:0)。4 拍后并行输出完整 4 位。

完整解答

【分析】4 个 D 触发器串联。每个时钟上升沿：新数据进入最左端，所有数据右移一位。

【逐拍分析 (si=1,0,1,1)】

拍	si	[Q3 Q2 Q1 Q0]
0	-	[0 0 0 0]
1	1	[1 0 0 0]
2	0	[0 1 0 0]
3	1	[1 0 1 0]
4	1	[1 1 0 1]

【VHDL】

```
1 entity sipo_4bit is
2   port (clk, rst_n, si : in std_logic;
3         po : out std_logic_vector(3 downto 0));
4 end sipo_4bit;
5
6 architecture behavioral of sipo_4bit is
7   signal reg : std_logic_vector(3 downto 0);
8 begin
9   process(clk, rst_n)
10    begin
```

```

11     if rst_n = '0' then reg <= (others => '0');
12     elsif rising_edge(clk) then
13         reg <= si & reg(3 downto 1);  -- 右移
14     end if;
15 end process;
16 po <= reg;
17 end behavioral;

```

【Testbench】

```

1 stimulus: process begin
2     rst_n <= '0'; si <= '0'; wait for 30 ns;
3     rst_n <= '1'; wait for 5 ns;
4     si <= '1'; wait for T; si <= '0'; wait for T;
5     si <= '1'; wait for T; si <= '1'; wait for T;
6     -- 4拍后po="1101"（注意：右移顺序反转）
7     wait;
8 end process;

```

3.2 题 2：并串转换（PISO）

题目：设计 4 位并入串出移位寄存器。先 load 数据，然后逐拍串行输出。

完整解答

【VHDL】

```

1 entity piso_4bit is
2     port (clk, rst_n, load : in std_logic;
3           data_in : in std_logic_vector(3 downto 0);
4           so : out std_logic);
5 end piso_4bit;
6
7 architecture behavioral of piso_4bit is
8     signal reg : std_logic_vector(3 downto 0);
9 begin
10    process(clk, rst_n)
11    begin
12        if rst_n = '0' then reg <= (others => '0');
13        elsif rising_edge(clk) then
14            if load = '1' then

```

```
15      reg <= data_in;  -- 并行装载
16      else
17          reg <= '0' & reg(3 downto 1);  -- 右移，高位补0
18      end if;
19  end if;
20 end process;
21 so <= reg(0);  -- 最低位串行输出
22 end behavioral;
```

【逐拍分析 (装载 data_in=1011)】

拍	操作	[Q3 Q2 Q1 Q0]
0	load	[1 0 1 1]
1	移位	[0 1 0 1] so=1
2	移位	[0 0 1 0] so=1
3	移位	[0 0 0 1] so=0
4	移位	[0 0 0 0] so=1

4 拍输出序列：1,1,0,1（刚好是原数据的倒序，因为右移先出最低位）。

3.3 题 3：双向移位寄存器

题目：设计 4 位双向移位寄存器。dir=1 右移，dir=0 左移。各有一个串行输入。

完整解答

【VHDL】

```
1  entity bidir_shift is
2      port (clk, rst_n, dir : in std_logic;
3            si_r, si_l : in std_logic;  -- 右移/左移串行输入
4            po : out std_logic_vector(3 downto 0));
5  end bidir_shift;
6
7  architecture behavioral of bidir_shift is
8      signal reg : std_logic_vector(3 downto 0);
9  begin
10     process(clk, rst_n)
11     begin
12         if rst_n = '0' then reg <= (others => '0');
13         elsif rising_edge(clk) then
```

```

14     if dir = '1' then
15         reg <= si_r & reg(3 downto 1);    -- 右移
16     else
17         reg <= reg(2 downto 0) & si_l;    -- 左移
18     end if;
19 end if;
20 end process;
21 po <= reg;
22 end behavioral;

```

【右移】数据从左边进入，向右移动 → 串行输入至少 4 拍后在 reg(0) 出现。【左移】数据从右边进入，向左移动 → 串行输入至少 4 拍后在 reg(3) 出现。

3.4 题 4: 74LS194 风格通用移位寄存器

题目：设计类似 74LS194 的通用移位寄存器：保持、右移、左移、并行装载。

完整解答

【模式控制】

S1	S0	功能
0	0	保持
0	1	右移
1	0	左移
1	1	并行装载

【VHDL】

```

1  entity shift_reg_194 is
2      port (clk, rst_n : in std_logic;
3            S : in std_logic_vector(1 downto 0);
4            si_r, si_l : in std_logic;
5            data_in : in std_logic_vector(3 downto 0);
6            po : out std_logic_vector(3 downto 0));
7  end shift_reg_194;
8
9  architecture behavioral of shift_reg_194 is
10     signal reg : std_logic_vector(3 downto 0);
11 begin
12     process(clk, rst_n)

```

```

13 begin
14     if rst_n = '0' then reg <= (others => '0');
15     elsif rising_edge(clk) then
16         case S is
17             when "00" => null;    -- 保持
18             when "01" => reg <= si_r & reg(3 downto 1);  -- 右移
19             when "10" => reg <= reg(2 downto 0) & si_l;  -- 左移
20             when "11" => reg <= data_in;  -- 装载
21         end case;
22     end if;
23 end process;
24 po <= reg;
25 end behavioral;

```

3.5 题 5：8 位移位寄存器

题目：设计 8 位串入并出移位寄存器。

完整解答

【分析】从 4 位扩展到 8 位，逻辑完全相同，仅位宽变化。

【VHDL】

```

1 entity sipo_8bit is
2     port (clk, rst_n, si : in std_logic; po : out std_logic_vector(7
3         downto 0));
4 end sipo_8bit;
5
6 architecture behavioral of sipo_8bit is
7     signal reg : std_logic_vector(7 downto 0);
8 begin
9     process(clk, rst_n)
10     begin
11         if rst_n = '0' then reg <= (others => '0');
12         elsif rising_edge(clk) then
13             reg <= si & reg(7 downto 1);  -- 8位右移
14         end if;
15     end process;
16     po <= reg;
17 end behavioral;

```


【推广】任意 N 位移位寄存器只需改变 type 宽度和移位拼接式的上界。

3.6 题 6: 循环移位寄存器

题目：设计循环右移寄存器。最低位移回最高位，数据不丢失。

完整解答

【VHDL】

```
1 entity rotate_right is
2   port (clk, rst_n : in std_logic; po : out std_logic_vector(7 downto 0)
3         );
4 end rotate_right;
5
6 architecture behavioral of rotate_right is
7   signal reg : std_logic_vector(7 downto 0) := "10000000";
8 begin
9   process(clk, rst_n)
10    begin
11      if rst_n = '0' then reg <= "10000000";
12      elsif rising_edge(clk) then
13        reg <= reg(0) & reg(7 downto 1); -- 最低位移到最高位!
14      end if;
15    end process;
16    po <= reg;
17 end behavioral;
```

【逐拍分析 (初始 10000000)】 10000000 → 01000000 → 00100000 → ... → 00000001 → 10000000

这其实就是环形计数器！循环移位 = 环形计数器的基础。

3.7 题 7: 算术右移寄存器

题目：设计算术右移寄存器（保持最高位符号位不变）。

完整解答

【VHDL】

```
1 entity arith_shift_right is
2   port (clk, rst_n : in std_logic; po : out std_logic_vector(7 downto 0)
3         );
4 end arith_shift_right;
5
6 architecture behavioral of arith_shift_right is
7   signal reg : std_logic_vector(7 downto 0);
8 begin
9   process(clk, rst_n)
10    begin
11      if rst_n = '0' then reg <= (others => '0');
12      elsif rising_edge(clk) then
13        reg <= reg(7) & reg(7 downto 1);  -- MSB保持!
14      end if;
15    end process;
16    po <= reg;
17 end behavioral;
```

【逻辑右移 vs 算术右移】

- 逻辑右移: `reg <= '0' & reg(7 downto 1)` — 高位补 0
- 算术右移: `reg <= reg(7) & reg(7 downto 1)` — 高位补符号位

【应用】 算术右移 1 位 = 有符号数 $\div 2$ （保持符号）。

3.8 题 8：移位寄存器实现 $\times 2$ 和 $\div 2$

题目：用移位寄存器实现无符号数的 $\times 2$ 和 $\div 2$ 。

完整解答**【左移 1 位 = $\times 2$ 】**

```
1 reg <= reg(6 downto 0) & '0';  -- 左移，低位补 0
```

【右移 1 位 = $\div 2$ （截断）】

```
1 reg <= '0' & reg(7 downto 1);  -- 右移，高位补 0
```

【验证】

- 0011(3) 左移 1 位 = 0110(6) = 3×2 □
- 0110(6) 右移 1 位 = 0011(3) = $6 \div 2$ □
- 0101(5) 右移 1 位 = 0010(2) = $5 \div 2 = 2$ (向下取整) □

3.9 题 9：移位寄存器序列检测方案

题目：用移位寄存器 + 比较器替代状态机检测序列 1011。

完整解答

【方案】4 位移位寄存器存储最近 4 位输入。当 reg="1011" 时，detected=1。

【VHDL】

```
1  entity shift_detector is
2      port (clk, rst_n, data_in : in std_logic; detected : out std_logic);
3  end shift_detector;
4
5  architecture rtl of shift_detector is
6      signal reg : std_logic_vector(3 downto 0);
7  begin
8      process(clk, rst_n)
9      begin
10         if rst_n = '0' then reg <= (others => '0');
11         elsif rising_edge(clk) then
12             reg <= data_in & reg(3 downto 1); -- 最新输入进最高位
13         end if;
14     end process;
15     detected <= '1' when reg = "1011" else '0';
16 end rtl;
```

【与状态机方案比较】

- 移位寄存器方案：代码极简，自动实现重叠检测
- 状态机方案：需要手动推导状态转移，但更灵活
- 考试中两者都要会——但移位寄存器方案通常更简单

3.10 题 10：延迟线

题目：用移位寄存器实现 N=5 拍的信号延迟（输入信号延迟 5 个时钟周期后输出）。

完整解答

【VHDL】

```
1 entity delay_line is
2     port (clk, rst_n, din : in std_logic; dout : out std_logic);
3 end delay_line;
4
5 architecture behavioral of delay_line is
6     signal reg : std_logic_vector(4 downto 0); -- 5级
7 begin
8     process(clk, rst_n)
9     begin
10         if rst_n = '0' then reg <= (others => '0');
11         elsif rising_edge(clk) then
12             reg <= din & reg(4 downto 1); -- 数据逐拍传递
13         end if;
14     end process;
15     dout <= reg(0); -- 输出=5拍前的输入
16 end behavioral;
```

【应用】数字信号处理中的流水线对齐、滤波器的抽头延迟线。

3.11 题 11：串行通信接收器

题目：设计 8 位串行数据接收器（1 起始位 +8 数据位，类似 UART 简版）。检测起始位后采集 8 位数据。

完整解答

【简化分析】8 位移位寄存器连续采 8 拍，并行输出。

【VHDL】

```
1 entity serial_rx is
2     port (clk, rst_n, rx : in std_logic;
3           data : out std_logic_vector(7 downto 0);
4           ready : out std_logic);
```

```

5  end serial_rx;
6
7  architecture behavioral of serial_rx is
8      signal reg : std_logic_vector(7 downto 0);
9      signal bit_cnt : integer range 0 to 7;
10     signal done : std_logic;
11 begin
12     process(clk, rst_n)
13     begin
14         if rst_n = '0' then
15             reg <= (others => '0'); bit_cnt <= 0; done <= '0';
16         elsif rising_edge(clk) then
17             reg <= rx & reg(7 downto 1);  -- 移位采集
18             if bit_cnt = 7 then
19                 bit_cnt <= 0; done <= '1';
20             else
21                 bit_cnt <= bit_cnt + 1; done <= '0';
22             end if;
23         end if;
24     end process;
25     data <= reg; ready <= done;
26 end behavioral;

```

3.12 题 12：给定初值和输入序列，推断寄存器状态

题目：4 位右移寄存器初值 0000，输入序列 1011。求 4 拍后的 reg 内容。

完整解答

【逐拍推导】

拍	si	reg
0	-	[0 0 0 0]
1	1	[1 0 0 0]
2	0	[0 1 0 0]
3	1	[1 0 1 0]
4	1	[1 1 0 1]

4 拍后 reg=[1101]。

【秒杀法】每个时钟：新输入进入左边，整体右移一位。4 拍后 reg 从高到低 = 最近 4 次输入（倒序）。

3.13 题 13：4 位 LFSR（伪随机序列）

题目：设计 4 位 LFSR，反馈多项式 $X^4 + X^3 + 1$ 。反馈位 = Q3 XOR Q2。

完整解答

【分析】反馈多项式 $X^4 + X^3 + 1 \rightarrow$ 反馈 = Q(4) XOR Q(3)（最高位和次高位异或）。状态数 $= 2^4 - 1 = 15$ （全 0 状态非法）。

【VHDL】

```

1 entity lfsr_4bit is
2   port (clk, rst_n : in std_logic; lfsr_out : out std_logic_vector(3
3     downto 0));
4 end lfsr_4bit;
5
6 architecture behavioral of lfsr_4bit is
7   signal reg : std_logic_vector(3 downto 0);
8   signal feedback : std_logic;
9 begin
10   feedback <= reg(3) xor reg(2); --  $X^4 + X^3 + 1$ 
11
12   process(clk, rst_n)
13   begin
14     if rst_n = '0' then reg <= "0001"; -- 非全0初值
15     elsif rising_edge(clk) then
16       reg <= feedback & reg(3 downto 1);
17     end if;
18   end process;
19   lfsr_out <= reg;
20 end behavioral;

```

【最大长度 LFSR 状态序列 (15 个)】

0001 $\rightarrow$ 1000 $\rightarrow$ 1100 $\rightarrow$ 1110 $\rightarrow$ 1111 $\rightarrow$ 0111 $\rightarrow$ 1011 $\rightarrow$ 0101

$\rightarrow$ 1010 $\rightarrow$ 1101 $\rightarrow$ 0110 $\rightarrow$ 0011 $\rightarrow$ 1001 $\rightarrow$ 0100 $\rightarrow$ 0010 $\rightarrow$ 0001

【应用】伪随机数生成、CRC 校验、扰码器。

3.14 题 14: 移位寄存器 Testbench 设计

题目: 为 8 位移位寄存器设计完整 Testbench, 全面验证所有功能。

完整解答

【Testbench】

```
1 entity shift8_tb is end entity;
2 architecture test of shift8_tb is
3     signal clk, rst_n, si : std_logic;
4     signal po : std_logic_vector(7 downto 0);
5     constant T : time := 20 ns;
6 begin
7     uut: entity work.sipo_8bit port map(clk, rst_n, si, po);
8     clk <= not clk after T/2;
9
10    process begin
11        rst_n <= '0'; si <= '0'; wait for 30 ns;
12        rst_n <= '1'; wait for 5 ns;
13
14        -- 测试1: 输入10110010
15        si <= '1'; wait for T; si <= '0'; wait for T;
16        si <= '1'; wait for T; si <= '1'; wait for T;
17        si <= '0'; wait for T; si <= '0'; wait for T;
18        si <= '1'; wait for T; si <= '0'; wait for T;
19        -- 8拍后po应 = "10110010" (注意顺序可能反转)
20
21        -- 测试2: 持续0输入看能否清空
22        si <= '0'; wait for 9*T;
23
24        -- 测试3: 持续1输入
25        si <= '1'; wait for 9*T; -- 8拍后po应全1
26        wait;
27    end process;
28 end test;
```

【验证清单】

1. 复位后 reg 全 0 ☐
2. 输入特定序列后 po 值正确 ☐
3. 连续 0 输入能清空寄存器 ☐

4. 连续 1 输入能填满寄存器 □

3.15 题 15：移位寄存器设计范式总结

完整解答

【四种基本模式的 VHDL 范式】

操作	VHDL
右移	reg <= si & reg(N-1 downto 1)
左移	reg <= reg(N-2 downto 0) & si
循环右移	reg <= reg(0) & reg(N-1 downto 1)
循环左移	reg <= reg(N-2 downto 0) & reg(N-1)
算术右移	reg <= reg(N-1) & reg(N-1 downto 1)
并行装载	if load='1' then reg <= data_in;

【关键考点】

- 串行输出 = reg(0)（右移）或 reg(N-1)（左移）
- 并行输出 = 整个 reg
- N 位移位寄存器 = N 个 D 触发器串联
- 数据从输入到输出延迟 = N 个时钟周期

第四部分

环形计数器变式题解析（15 题）

第四章 环形计数器变式题（15 题完整解析）

4.1 题 1：4 位环形计数器

题目：设计 4 位环形计数器。初始 1000。状态：1000→0100→0010→0001→1000。

完整解答

【结构】4 位移位寄存器，Q0 反馈回 D3（右移 + 反馈）。

【逐拍分析】

拍	Q3	Q2	Q1	Q0
0(初)	1	0	0	0
1	0	1	0	0
2	0	0	1	0
3	0	0	0	1
4	1	0	0	0（循环!）

【VHDL】

```
1 entity ring4 is
2     port (clk, rst_n : in std_logic; q : out std_logic_vector(3 downto 0))
3         ;
4 end ring4;
5
6 architecture behavioral of ring4 is
7     signal reg : std_logic_vector(3 downto 0);
8 begin
9     process(clk, rst_n)
10    begin
11        if rst_n = '0' then reg <= "1000"; -- One-hot 初始
12        elsif rising_edge(clk) then
13            reg <= reg(0) & reg(3 downto 1); -- 右移+反馈
```

```

13     end if;
14     end process;
15     q <= reg;
16 end behavioral;

```

【为什么是 4 个状态】 4 个 D 触发器，只有 1 个 1 在循环移动。1 在 4 个位置 → 4 个状态。

【关键 VHDL 语句】 `reg <= reg(0) & reg(3 downto 1)`

- `reg(0)→reg(3)`: 最低位反馈到最高位（闭环!）
- `reg(3)→reg(2)`: 原最高位右移一位
- `reg(2)→reg(1)`: 右移
- `reg(1)→reg(0)`: 右移

4.2 题 2: 8 位环形计数器

题目：设计 8 位环形计数器（8 个状态）。

完整解答

【VHDL】

```

1  entity ring8 is
2      port (clk, rst_n : in std_logic; q : out std_logic_vector(7 downto 0))
3          ;
4  end ring8;
5
6  architecture behavioral of ring8 is
7      signal reg : std_logic_vector(7 downto 0);
8  begin
9      process(clk, rst_n)
10     begin
11         if rst_n = '0' then reg <= "10000000";
12         elsif rising_edge(clk) then
13             reg <= reg(0) & reg(7 downto 1); -- 8位环形
14         end if;
15     end process;
16     q <= reg;

```

16end behavioral;

【状态循环】 10000000 → 01000000 → ⋯ → 00000001 → 10000000
共 8 个状态（8 拍一个循环）。

4.3 题 3：4 位约翰逊计数器（扭环形）

题目:设计 4 位约翰逊计数器。初始 0000。状态序列:0000→1000→1100→1110→1111→0111→0011→

完整解答

【反馈】 $\overline{Q_0} \rightarrow D3$ （反相最低位反馈回最高位）。

【逐拍分析】

拍	Q3	Q2	Q1	Q0
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1
8	0	0	0	0（循环!）

【VHDL】

1entity johnson4 is
2port (clk, rst_n : in std_logic; q : out std_logic_vector(3 downto 0))
3;end johnson4;
4
5architecture behavioral of johnson4 is
6signal reg : std_logic_vector(3 downto 0);
7begin
8process(clk, rst_n)
9begin
10if rst_n = '0' then reg <= (others => '0');
11elseif rising_edge(clk) then

12 reg <= (not reg(0)) & reg(3 downto 1); -- 反相反馈!
13 end if;
14 end process;
15 q <= reg;
16 end behavioral;

【关键 VHDL】 reg <= (not reg(0)) & reg(3 downto 1)
注意 not reg(0) 一反相！这是约翰逊与普通环形计数器的唯一区别。

【状态数】 4 个 DFF→ $2 \times 4 = 8$ 个状态。比普通环形计数器多一倍。

4.4 题 4： 环形计数器 vs 二进制计数器

题目：对比 4 状态环形计数器与 2 位二进制计数器。

完整解答

特性	4 位环形 (4 状态)	2 位二进制 (4 状态)
DFF 数量	4	2
状态编码	One-hot(1000,0100,...)	Binary(00,01,10,11)
解码门数	0 (读 Qx 即可)	需要 2-4 译码器
关键路径延迟	1 级 DFF(t_{cko})	t_{cko} + 进位链 + t_{setup}
最大频率	最高 (D 触发器本身限制)	较低 (进位链限制)
面积	4×DFF	2×DFF + 加法器

【选择规则】 需要高速时选环形计数器；面积受限时选二进制计数器。

4.5 题 5： 自启动环形计数器

题目：4 位环形计数器可能进入非法状态。设计自启动逻辑。

完整解答

【非法状态检测】 合法状态：one-hot（恰好一个 1）。非法状态：全 0、多个 1。

【VHDL——自启动】

```

1  entity ring4_selfstart is
2      port (clk, rst_n : in std_logic; q : out std_logic_vector(3 downto 0))
3      ;
4  end ring4_selfstart;
5
6  architecture behavioral of ring4_selfstart is
7      signal reg : std_logic_vector(3 downto 0);
8      function count_ones(v : std_logic_vector) return integer is
9          variable c : integer := 0;
10         begin
11             for i in v'range loop if v(i)='1' then c:=c+1; end if; end loop;
12             return c;
13         end function;
14     begin
15         process(clk, rst_n)
16         begin
17             if rst_n = '0' then reg <= "1000";
18             elsif rising_edge(clk) then
19                 if count_ones(reg) /= 1 then -- 非法状态检测
20                     reg <= "1000"; -- 强制恢复
21                 else
22                     reg <= reg(0) & reg(3 downto 1);
23                 end if;
24             end if;
25         end process;
26         q <= reg;
27     end behavioral;

```

4.6 题 6：自启动约翰逊计数器

题目：约翰逊计数器也可能进入非法状态。设计自启动逻辑。

完整解答

【分析】4 位约翰逊计数器合法状态 =8 个。非法状态 =8 个（全 16 个状态的一半）。检测不在合法状态中 → 强制恢复。

【VHDL——预存合法状态 ROM】

```

1  type rom_type is array (0 to 7) of std_logic_vector(3 downto 0);
2  constant LEGAL : rom_type := (

```

```

3  "0000","1000","1100","1110","1111","0111","0011","0001"
4  );
5  signal is_legal : boolean;
6  begin
7      is_legal <= false;
8      for i in 0 to 7 loop
9          if reg = LEGAL(i) then is_legal <= true; end if;
10     end loop;
11
12     if not is_legal then
13         reg <= "0000"; -- 强制恢复
14     else
15         reg <= (not reg(0)) & reg(3 downto 1);
16     end if;

```

4.7 题 7：用环形计数器做序列发生器

题目：用 4 位环形计数器输出 Q3 产生序列。Q3 序列是什么？

完整解答

【逐拍分析 Q3】

拍	Q3
0	1
1	0
2	0
3	0
4	1
5	0
...	...

Q3 输出序列：1, 0, 0, 0, 1, 0, 0, 0, ... 一周期 4。

【各 Q 输出对比】

- Q3: 1,0,0,0,1,0,0,0,... (相位 0°)
- Q2: 0,1,0,0,0,1,0,0,... (相位 90°)

• Q1: 0,0,1,0,0,0,1,0,...（相位 180° ）

• Q0: 0,0,0,1,0,0,0,1,...（相位 270° ）

四个 Q 输出正好是相位差 90° 的 4 路信号——可用于多相时钟生成。

4.8 题 8：左移环形计数器

题目：设计左移环形计数器（1 从右向左移动）。

完整解答

【VHDL】

```
reg <= reg(2 downto 0) & reg(3); -- 左移循环
```

【状态序列 (初始 0001)】 0001 $\rightarrow$ 0010 $\rightarrow$ 0100 $\rightarrow$ 1000 $\rightarrow$ 0001

1 从右向左循环移动。

4.9 题 9：N 位环形计数器状态数推导

题目：N 位移位寄存器构建环形计数器，能产生多少个状态？

完整解答

【普通环形计数器】 One-hot 编码。初始化后恰好一个 1。N 个位置 $\rightarrow$ N 个状态。

答案：N 个状态。

【约翰逊计数器】 反相反馈。N 个 DFF $\rightarrow$ $2N$ 个不同状态。

答案： $2N$ 个状态。

【证明（约翰逊）】 每个 DFF 在循环中都经历 0 和 1 的完整变化。反相反馈使得 1 连续填入，0 连续推出。每 N 拍状态翻转一次，共 $2N$ 拍回到起点。

4.10 题 10：5 位环形计数器时序控制器

题目：用 5 位环形计数器实现 5 步时序控制器。Q4-Q0 分别作为步骤 1-5 的使能信号。

完整解答

【VHDL】

```
1 entity seq_controller is
2     port (clk, rst_n : in std_logic;
3           step : out std_logic_vector(4 downto 0));
4 end seq_controller;
5
6 architecture behavioral of seq_controller is
7     signal reg : std_logic_vector(4 downto 0);
8 begin
9     process(clk, rst_n)
10    begin
11        if rst_n = '0' then reg <= "10000"; -- 步骤1激活
12        elsif rising_edge(clk) then
13            reg <= reg(0) & reg(4 downto 1);
14        end if;
15    end process;
16    step <= reg;
17 end behavioral;
```

【应用】多周期操作的控制器（如乘法器的状态控制）。每拍恰好一个步骤激活。

4.11 题 11：给定波形判断计数器类型

题目：4 位输出波形：1000→1100→1110→1111→0111→0011→0001→0000→1000→…。
判断类型和状态数。

完整解答

【分析】

- 状态数=8（不同模式有 8 种）
- 观察规律：1 从左向右填充，然后 0 从右向左清空
- 这是典型的约翰逊计数器！

【结论】4 位约翰逊计数器（8 个状态）。

【判断方法】

- One-hot（只有一个 1 循环）= 普通环形
- 111...→011...→001... 模式 = 约翰逊（扭环形）
- 连续递增的二进制值 = 二进制计数器

4.12 题 12：LFSR vs 环形计数器

题目：比较 LFSR 和环形计数器的异同。

完整解答

	LFSR	环形计数器
结构	移位寄存器 +XOR 反馈	移位寄存器 + 直接反馈
反馈逻辑	多输入异或（抽头）	单输入（reg(0) 或 reg(0)）
状态数 (N 个 DFF)	最多 $2^N - 1$	最多 2N
状态编码	伪随机	One-hot
应用	随机数、CRC	时序控制、多相信号

4.13 题 13：约翰逊计数器做分频器

题目：4 位约翰逊计数器（8 状态）。Q3 输出频率？

完整解答

【分析】Q3 在 8 拍中：0→1→1→1→1→0→0→0→0→…。周期 =8 拍。

$$f_{Q3} = f_{clk}/8$$

【各 Q 频率（4 位约翰逊）】所有 Q 输出的频率都是 $f_{clk}/8$ ，但相位不同。

【与二进制计数器对比】

- 二进制 Q0：每 2 拍翻转 → $f_{clk}/2$
- 二进制 Q1：每 4 拍翻转 → $f_{clk}/4$

- 二进制 Q2: 每 8 拍翻转 $\rightarrow f_{clk}/8$
- 约翰逊任意 Q: 每 8 拍翻转 $\rightarrow f_{clk}/8$ (所有 Q 频率相同)

4.14 题 14: 任意 N 位环形计数器模板

题目: 写出 N 位环形计数器的通用 VHDL 模板。

完整解答

【通用模板】

```

1  entity ring_counter is
2      generic (N : integer := 4);
3      port (clk, rst_n : in std_logic; q : out std_logic_vector(N-1 downto
4              0));
5  end ring_counter;
6
7  architecture behavioral of ring_counter is
8      signal reg : std_logic_vector(N-1 downto 0);
9  begin
10     process(clk, rst_n)
11     begin
12         if rst_n = '0' then
13             reg <= (N-1 => '1', others => '0'); -- 只有MSB=1
14         elsif rising_edge(clk) then
15             reg <= reg(0) & reg(N-1 downto 1); -- 右移+反馈
16         end if;
17     end process;
18     q <= reg;
19 end behavioral;

```

【参数化】 N 可以是任意值。 $N=4 \rightarrow 4$ 个状态。 $N=8 \rightarrow 8$ 个状态。 N 个 DFF $\rightarrow N$ 个状态。

4.15 题 15：环形计数器类统一秒杀

环形计数器秒杀总结

普通环形计数器：

- VHDL: `reg <= reg(0) & reg(N-1 downto 1)`
- N 个 DFF $\rightarrow$ N 个状态
- 初始：恰好一个 1（One-hot）
- 每拍：1 向右移一位（通过反馈形成循环）

约翰逊（扭环形）计数器：

- VHDL: `reg <= (not reg(0)) & reg(N-1 downto 1)`
- N 个 DFF $\rightarrow$ 2N 个状态
- 初始：全 0（0000）
- 规律：1 从左填满，0 从右清空

口诀：

- 环形 = 1 在环里跑，跑 N 步回原点
- 约翰逊 = 1 填满再清空，跑 2N 步回原点

第五部分

序列检测器变式题解析（20 题）

第五章 序列检测器变式题（20 题完整解析）

5.1 题 1：检测 1011（Moore 型，非重叠）

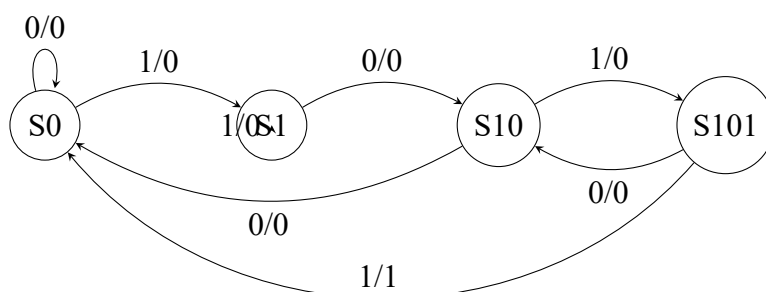
题目：设计 Moore 型状态机，检测 1011（非重叠）。画状态图，写 VHDL，写 Testbench。

完整解答

【状态定义】 S0: 未匹配（0 位）。S1: 已匹配“1”。S10: 已匹配“10”。S101: 已匹配“101”。

检测条件：S101 状态 + 输入 = 1（非重叠 → 回 S0）。

【状态转移图】



【状态转移表】

当前状态	输入	下一状态	输出
S0	0	S0	0
S0	1	S1	0
S1	0	S10	0
S1	1	S1	0
S10	0	S0	0
S10	1	S101	0
S101	0	S10	0
S101	1	S0	1

【VHDL (Moore 型)】

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity detector_1011_moore is
5      port (clk, rst_n, din : in std_logic; detected : out std_logic);
6  end detector_1011_moore;
7
8  architecture fsm of detector_1011_moore is
9      type state_type is (S0, S1, S10, S101);
10     signal state, next_state : state_type;
11 begin
12     -- 状态寄存器
13     process(clk, rst_n)
14     begin
15         if rst_n = '0' then state <= S0;
16         elsif rising_edge(clk) then state <= next_state; end if;
17     end process;
18
19     -- 次态逻辑
20     process(state, din)
21     begin
22         case state is
23             when S0 =>
24                 if din = '1' then next_state <= S1; else next_state <= S0; end
25                 if;
26             when S1 =>
27                 if din = '0' then next_state <= S10; else next_state <= S1; end
28                 if;
29             when S10 =>
30                 if din = '1' then next_state <= S101; else next_state <= S0; end
31                 if;
32             when S101 =>
33                 if din = '1' then next_state <= S0; -- 非重叠
34                 else next_state <= S10; end if;
35             end case;
36         end process;
37
38     -- 输出逻辑(Moore: 仅取决于状态)
39     detected <= '1' when (state = S101 and din = '1') else '0';
40 end fsm;

```

【Testbench】

```

1 entity detector_tb is end entity;
2 architecture test of detector_tb is
3     signal clk, rst_n, din, detected : std_logic;
4     constant T : time := 20 ns;
5 begin
6     uut: entity work.detector_1011_moore port map(clk, rst_n, din,
7         detected);
8     clk <= not clk after T/2;
9
10    process begin
11        rst_n <= '0'; din <= '0'; wait for 30 ns;
12        rst_n <= '1'; wait for 5 ns;
13        -- 输入: 1 0 1 1 0 1 0 1 1
14        din <= '1'; wait for T; din <= '0'; wait for T;
15        din <= '1'; wait for T; din <= '1'; wait for T; -- 第4拍 → detected
16        !
17        din <= '0'; wait for T; din <= '1'; wait for T;
18        din <= '0'; wait for T; din <= '1'; wait for T;
19        din <= '1'; wait for T; -- 第9拍 → detected again!
20        wait;
21    end process;
22 end test;

```

【波形分析】

拍	din	状态	detected
1	1	S0→S1	0
2	0	S1→S10	0
3	1	S10→S101	0
4	1	S101→S0	1
5	0	S0→S0	0
6	1	S0→S1	0
7	0	S1→S10	0
8	1	S10→S101	0
9	1	S101→S0	1

第 4 拍和第 9 拍检测到 1011。第 4 拍后的 0（非 1）使状态机回到 S0——非重叠检测，正确！

5.2 题 2：检测 1011（重叠检测）

题目：设计检测 1011 的**重叠**检测器。重叠意味着：检测到后，最后的”1”可以作为新序列的第一个”1”。

完整解答

【关键区别】 S101 + 输入 = 1 → 检测到！ → 去 S1（不是 S0！）

最后的 1 作为新序列的第一个 1。

【修改的转移（仅改一行）】

```

1 when S101 =>
2   if din = '1' then
3     next_state <= S1;  -- 重叠！ → 去 S1（最后的 1 是新序列的第一个 1）
4   else
5     next_state <= S10;
6   end if;

```

【波形对比（输入 1011011）】

拍	输入	非重叠检测	重叠检测
1-4	1,0,1,1	检测在第 4 拍	检测在第 4 拍
4	1(检测到)	回 S0	去 S1
5	0	S0(等 1)	S10
6	1	S1	S101
7	1	S1	第 7 拍又检测到！

重叠检测在 7 拍内检测到 2 次。非重叠只检测到 1 次。

【考试经常考这个区别！】

5.3 题 3：检测 1101（重叠）

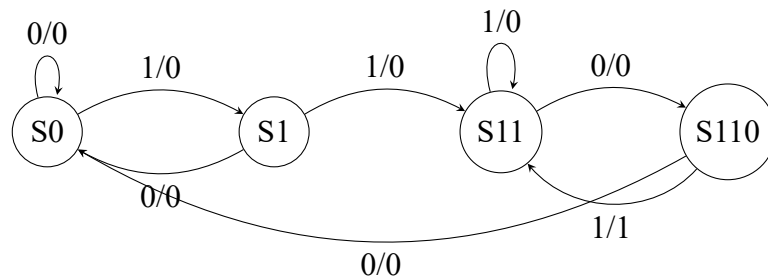
题目：设计重叠检测 1101 的状态机。

完整解答

【状态定义】 S0→S1(已匹配 1)→S11(已匹配 11)→S110(已匹配 110)

检测：S110 + 输入 = 1。重叠：最后 1→S1。

【状态转移图】



【转移表】

当前	din	次态	输出
S0	0	S0	0
S0	1	S1	0
S1	0	S0	0
S1	1	S11	0
S11	0	S110	0
S11	1	S11	0
S110	0	S0	0
S110	1	S11(重叠)	1

【VHDL 关键转移】

```

1 when S110 =>
2   if din = '1' then next_state <= S11;  -- 重叠：最后1→S11
3   else next_state <= S0; end if;

```

【关键分析】S11+1→S11（“111”中后两个1可作为新的“11”前缀）。S110+1→检测到→S11（重叠！“1101”最后的“1”→1→“11”中）。

5.4 题 4：检测 0101（重叠）

题目：设计重叠检测 0101 的状态机。

完整解答

【状态】S0→S01(0)→S010(01)→S0101(检测)→重叠时去 S01

【转移表】

当前	din	次态	输出
S0	0	S01	0
S0	1	S0	0
S01	0	S01	0
S01	1	S010	0
S010	0	S0101	0
S010	1	S0	0
S0101	0	S01	0
S0101	1	S010(重叠)	1

【重叠行为】 S0101+1→检测到。最后的”1”→新序列的第二步（需要前面有 0，所以去 S01(0 已在前一步提供了)）。

不对——仔细分析：S0101+0→S01（”0” 可以是新序列的第一个 0）
S0101+1→S010（”1” 匹配了序列的第二步”01”中的”1”）

5.5 题 5: 检测 1111 (连续 4 个 1)

题目：设计重叠检测 1111 (连续 4 个 1) 的状态机。

完整解答

【状态】 S0→S1(1 个 1)→S11(2 个 1)→S111(3 个 1)。S111+1→检测到。

【关键】 重叠：任何时候输入 0→回 S0。但 S111+1→去 S111（”1111”中后 3 个 1 可作为新”111”前缀）。

【VHDL】

```

1 type state_type is (S0, S1, S11, S111);
2 ...
3 when S0 =>
4   if din = '1' then next_state <= S1; else next_state <= S0; end if;
5 when S1 =>
6   if din = '1' then next_state <= S11; else next_state <= S0; end if;
7 when S11 =>
8   if din = '1' then next_state <= S111; else next_state <= S0; end if;
9 when S111 =>
10  if din = '1' then
11    next_state <= S111; -- 重叠! 后3个1仍是有效前缀
12  else

```

```

13   next_state <= S0;
14   end if;

```

【输出】 detected = '1' when state=S111 and din='1'。

5.6 题 6：检测 1001（重叠）

题目：设计重叠检测 1001 的状态机。

完整解答

【状态】 S0→S1(1)→S10(10)→S100(100)。S100+1→检测到→重叠去 S1（最后的 1 是新序列的第一个 1）。

【关键转移】

- S100+0→S0（"1000" 不匹配任何有效前缀）
- S100+1→检测到！→S1（重叠：最后的 1→新序列第一步）

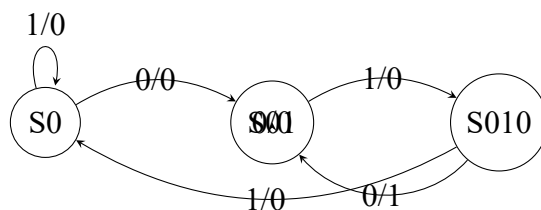
5.7 题 7：检测 010（3 位序列）

题目：设计重叠检测 010 的状态机（3 位序列）。

完整解答

【状态】 S0→S01(0)→S010(检测)。重叠行为：S010+0→S01（最后的 0→新序列第 1 位）。

【转移图】



【VHDL】

```

1 type state_type is (S0, S01, S010);
2 signal state : state_type;
3 begin
4     process(clk, rst_n)
5     begin
6         if rst_n = '0' then state <= S0;
7         elsif rising_edge(clk) then
8             case state is
9                 when S0 =>
10                 if din = '0' then state <= S01; end if;
11                 when S01 =>
12                 if din = '1' then state <= S010; end if;
13                 when S010 =>
14                 if din = '0' then state <= S01;    -- 重叠!
15                 else state <= S0; end if;
16             end case;
17         end if;
18     end process;
19     detected <= '1' when (state = S010 and din = '0') else '0';
20 end fsm;

```

5.8 题 8: 检测 101 (3 位序列)

题目: 设计重叠检测 101 的状态机。

完整解答

【状态】 S0→S1(1)→S10(10)。S10+1→检测到→去 S1 (最后的 1→新序列的第一个 1, 重叠)。

【注意对比】 检测 101 非重叠: S10+1→检测到→去 S0。

5.9 题 9: 同时检测 1011 和 1101 (合并状态机)

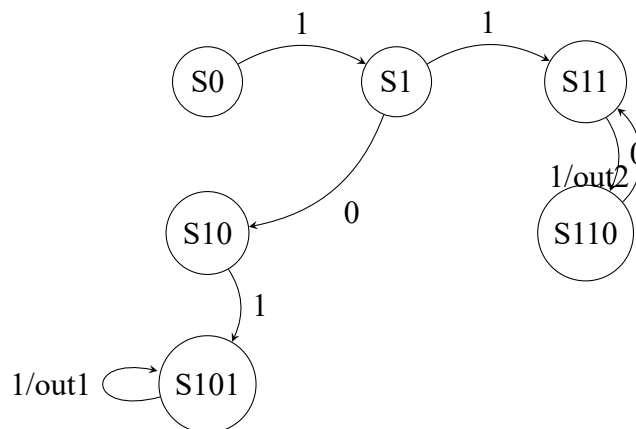
题目: 设计能同时检测 1011 和 1101 的状态机。任一检测到, 对应输出 =1。

完整解答

【状态共享】两个序列共享前缀“1”：

- $S0 \rightarrow S1(1) \rightarrow$ 分支：S10(10: 1011 的前缀) 或 S11(11: 1101 的前缀)
- $S10 \rightarrow S101(101) \rightarrow (1 \rightarrow \text{检测到 } 1011)$
- $S11 \rightarrow S110(110) \rightarrow (1 \rightarrow \text{检测到 } 1101)$

【状态图】



【VHDL 关键转移】

```

1  when S0 =>
2    if din = '1' then state <= S1; end if;
3  when S1 =>
4    if din = '0' then state <= S10;
5    elsif din = '1' then state <= S11; end if;
6  when S10 =>
7    if din = '1' then state <= S101; else state <= S0; end if;
8  when S11 =>
9    if din = '0' then state <= S110;
10   elsif din = '1' then state <= S11; end if;
11 when S101 =>
12   if din = '1' then state <= S1;    -- 检测到1011!
13   elsif din = '0' then state <= S10; end if;
14 when S110 =>
15   if din = '1' then state <= S11;  -- 检测到1101!
16   elsif din = '0' then state <= S0; end if;
  
```

5.10 题 10：检测序列长度可配置（移位寄存器方案）

题目：用移位寄存器 + 比较器实现可配置的序列检测器。目标序列 TARGET 可动态设置。

完整解答

【VHDL】

```
1 entity config_detector is
2   port (clk, rst_n, din : in std_logic;
3         TARGET : in std_logic_vector(3 downto 0);
4         detected : out std_logic);
5 end config_detector;
6
7 architecture rtl of config_detector is
8   signal reg : std_logic_vector(3 downto 0);
9 begin
10  process(clk, rst_n)
11  begin
12    if rst_n = '0' then reg <= (others => '0');
13    elsif rising_edge(clk) then
14      reg <= din & reg(3 downto 1);
15    end if;
16  end process;
17  detected <= '1' when reg = TARGET else '0';
18 end rtl;
```

【优点】无需修改状态机，改变 TARGET 即可检测任何 4 位序列。

【局限】自动重叠检测（移位寄存器天然支持重叠）。

5.11 题 11：统计检测次数

题目：扩展 1011 检测器，增加计数器统计检测到的总次数（0-255）。

完整解答

【VHDL——在检测信号上加计数器】

```
1 signal detect_count : integer range 0 to 255;
2 signal detected : std_logic;
```

```
3
4  -- 原有状态机代码...
5  detected <= '1' when (state = S101 and din = '1') else '0';
6
7  -- 检测计数器
8  process(clk, rst_n)
9  begin
10     if rst_n = '0' then detect_count <= 0;
11     elsif rising_edge(clk) then
12         if detected = '1' then
13             detect_count <= detect_count + 1;
14         end if;
15     end if;
16 end process;
```

5.12 题 12：带超时检测的序列检测器

题目：若超过 16 拍未检测到序列，输出 timeout。

完整解答

【VHDL】

```
1  signal timeout_cnt : integer range 0 to 15;
2  ...
3  process(clk, rst_n)
4  begin
5      if rst_n = '0' then timeout_cnt <= 0;
6      elsif rising_edge(clk) then
7          if detected = '1' then
8              timeout_cnt <= 0; -- 检测到→清超时计数
9          elsif state = S1 then -- 匹配进行中→保持计数
10             timeout_cnt <= timeout_cnt;
11         else
12             if timeout_cnt = 15 then
13                 timeout_cnt <= 15; -- 保持超时
14             else
15                 timeout_cnt <= timeout_cnt + 1;
16             end if;
17         end if;
```



```

18     end if;
19 end process;
20 timeout <= '1' when timeout_cnt = 15 else '0';

```

5.13 题 13: Moore 型 vs Mealy 型对比

题目：分别写出检测 1011 的 Moore 型和 Mealy 型状态机，比较异同。

完整解答

	Moore 型	Mealy 型
输出决定因素	仅当前状态	状态 + 输入
状态数	多 (需“检测到”状态)	少 (检测在转移上)
检测延迟	晚 1 拍	同 1 拍
输出稳定性	全周期稳定	输入变化时可能抖
考试推荐	更安全	更快

【Mealy 型 VHDL (更简洁)】

```

1  -- Mealy: 输出在转移中表达
2  when S101 =>
3      if din = '1' then
4          next_state <= S1; detected <= '1';  -- Mealy 输出
5      else
6          next_state <= S10; detected <= '0';
7      end if;

```

【推荐】考试如无特别说明，用 Moore 型。更稳定，状态图更清晰。

5.14 题 14: 4 位并行序列检测

题目：输入是 4 位并行数据，检测特定 4 位模式（如“1011”）。每拍判断一次。

完整解答

【分析】纯组合逻辑！不需要状态机。 $detected = '1' \text{ when } data_in = "1011"$ 。

【VHDL】

```

1 entity parallel_detect is
2     port (data_in : in std_logic_vector(3 downto 0);
3           detected : out std_logic);
4 end parallel_detect;
5
6 architecture rtl of parallel_detect is
7 begin
8     detected <= '1' when data_in = "1011" else '0';
9 end rtl;

```

【与串行检测的本质区别】 并行检测不需要记忆历史——当前 4 位数据同时可见。串行检测需要状态机或移位寄存器来“记住”历史。

5.15 题 15：检测包含“无关位”的序列

题目：检测 1xx1（第 1 位和第 4 位为 1，中间两位任意）。

完整解答

【移位寄存器方案】

```

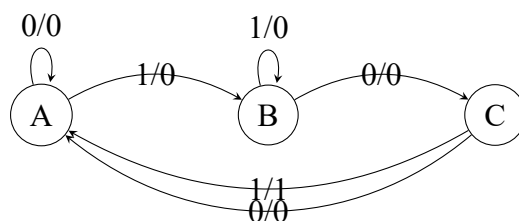
1 detected <= '1' when reg(3) = '1' and reg(0) = '1' else '0';
2 -- 只检查第1位和第4位

```

【状态机方案】 状态 S1→S1x→S1xx→S1xx1(检测)。中间两位任意值都接受。

5.16 题 16：给定状态图写 VHDL

题目：给定下面状态图，写出完整 VHDL 状态机。



完整解答

【从图到代码的映射】箭头标注 X/Y = 输入/输出。

【VHDL】

```

1 type state_type is (A_state, B_state, C_state);
2 signal state : state_type;
3 begin
4   process(clk, rst_n)
5   begin
6     if rst_n = '0' then state <= A_state;
7     elsif rising_edge(clk) then
8       case state is
9         when A_state =>
10          if din = '1' then state <= B_state; end if;
11          -- din='0': 保持A_state (loop above)
12         when B_state =>
13          if din = '0' then state <= C_state; end if;
14          -- din='1': 保持B_state (loop above)
15         when C_state =>
16          state <= A_state; -- 无论0还是1都回A
17       end case;
18     end if;
19   end process;
20   detected <= '1' when (state = C_state and din = '1') else '0';
21 end fsm;

```

5.17 题 17: 给定输入序列和状态机, 求输出序列

题目: 1011 检测器 (重叠), 输入 = 1011011。求输出序列。

完整解答

【逐拍追踪】

拍	1	2	3	4	5	6	7
din	1	0	1	1	0	1	1
状态 (当前)	S0	S1	S10	S101	S1	S10	S101
状态 (次态)	S1	S10	S101	S1	S10	S101	S1
输出	0	0	0	1	0	0	1

输出序列：0001001

- 第 4 拍：S101+1→检测 1011！→重叠去 S1
- 第 7 拍：S101+1→再次检测 1011！→重叠去 S1

5.18 题 18：状态编码优化

题目：1011 检测器 4 个状态。分别用 Binary(2 位) 和 One-hot(4 位) 编码。哪个体积更小？哪个速度更快？

完整解答

【Binary 编码（2 位 DFF）】

状态	编码
S0	00
S1	01
S10	10
S101	11

优点：2 个 DFF（面积小）。缺点：需要译码电路（速度慢）。

【One-hot 编码（4 位 DFF）】

状态	编码
S0	0001
S1	0010
S10	0100
S101	1000

优点：无需译码（速度快）。缺点：4 个 DFF（面积大）。

【考试答案】面积最小 →Binary。速度最快 →One-hot。

【FPGA 实际】综合工具默认自动选择最优编码，你不需要手动指定。

5.19 题 19：序列检测器模板化

题目：编写检测任意 4 位序列的通用 VHDL 模板（参数化：TARGET 可配置）。

完整解答

【状态机通用模板】

```

1  entity seq_detector_generic is
2      generic (TARGET : std_logic_vector(3 downto 0));
3      port (clk, rst_n, din : in std_logic; detected : out std_logic);
4  end seq_detector_generic;
5
6  architecture fsm of seq_detector_generic is
7      type state_type is (S0, S1, S2, S3);
8      signal state : state_type;
9  begin
10     process(clk, rst_n)
11     begin
12         if rst_n = '0' then state <= S0;
13         elsif rising_edge(clk) then
14             case state is
15                 when S0 =>
16                     if din = TARGET(3) then state <= S1; end if;
17                 when S1 =>
18                     if din = TARGET(2) then state <= S2;
19                     elsif din = TARGET(3) then state <= S1;
20                     else state <= S0; end if;
21                 when S2 =>
22                     if din = TARGET(1) then state <= S3;
23                     elsif din = TARGET(3) then state <= S1;
24                     else state <= S0; end if;
25                 when S3 =>
26                     if din = TARGET(0) then state <= S1;  -- 检测+重叠
27                     elsif din = TARGET(3) then state <= S1;
28                     else state <= S0; end if;
29             end case;
30         end if;
31     end process;
32     detected <= '1' when (state = S3 and din = TARGET(0)) else '0';
33 end fsm;

```

【使用】 generic map(TARGET=>"1011") 即可检测 1011。

5.20 题 20：序列检测器秒杀总结

序列检测器——考试必考，20 题统一解法

第一步：状态 = 已匹配的前缀

- 每个状态代表”已匹配了目标序列的前几位”
- N 位序列 → 最多 N 个状态（不含初始 S0）

第二步：画状态转移图对每个状态，考虑输入 0 和输入 1 分别：

- 匹配推进：去下一个状态
- 匹配失败：找最长的”已匹配后缀”作为新状态
- 检测到：输出 =1

第三步：确定重叠/非重叠

- 重叠：检测到后 → 检查最后的输入能否构成新前缀
- 非重叠：检测到后 → 回 S0

第四步：写 VHDL（三段式 Moore 型）

1. 状态寄存器：process(clk,rst) + rising_edge
2. 次态逻辑：process(state,din) + case
3. 输出逻辑：并行信号赋值

常考序列状态数速查：

- 1011 → 4 状态 (S0,S1,S10,S101)
- 1101 → 4 状态 (S0,S1,S11,S110)
- 1111 → 4 状态 (S0,S1,S11,S111)
- 0101 → 4 状态 (S0,S01,S010,S0101)
- 101 → 3 状态 (S0,S1,S10)
- 010 → 3 状态 (S0,S01,S010)

VHDL 状态机模板（一行不改，只换状态名和转移逻辑）

```
1 type state_type is (...);
2 signal state : state_type;
3 process(clk, rst_n)
4 begin
5     if rst_n='0' then state <= S0;
6     elsif rising_edge(clk) then
7         case state is
8             when S0 => if din='1' then state <= S1; end if;
9             ...
10        end case;
11    end if;
12 end process;
13 detected <= '1' when (...) else '0';
```

速胜法宝：移位寄存器替代方案序列长度 ≤ 8 位时，用移位寄存器 + 比较器实现序列检测比状态机简单得多！

```
1 reg <= din & reg(N-1 downto 1);
2 detected <= '1' when reg = TARGET else '0'; -- 一行搞定！
```

但注意：考试如果明确要求“用状态机设计”，则必须用状态机！